Laboratory Activity# 3: Role-Based Login System with REST API CRUD
Subject: System Integration and Architecture

Name: NAZA,ANGELICA M.
Course/Year: BSIT-3E

I. Activity Overview

In this laboratory activity, you will build a mock role-based login system using **Node.js** and **Express**. The system stores users **in memory** (no database), allows accounts to be created via **Postman**, and redirects users to different dashboards based on their role (**admin**, **staff**, or **user**). You will demonstrate complete **CRUD operations** (Create, Read, Update, Delete) through REST API endpoints and then integrate the backend with a web-based login page that consumes the API.
This lab demonstrates how REST APIs function as the backbone of modern web applications, handling synchronous communication between client and server using JSON data format.

II. Learning Objectives

At the end of this activity, you should be able to:

1. **LO1** – Create an Express REST API with full CRUD operations for a users resource.
2. **LO2** – Add a login endpoint (POST /api/login) that validates credentials and returns a role-based redirect.
3. **LO3** – Create an HTML login page that consumes the login API and follows the server's redirect decision.
4. **LO4** – Build protected dashboards (admin, staff, user) that use sessionStorage for session tracking.
5. **LO5** – Test the API using Postman and the login flow using a browser.
6. **LO6** – Explain the difference between client-side and server-side authentication decisions.
7. **LO7** – Demonstrate how GET, POST, PUT, and DELETE methods support REST API integration.

III. Pre-Lab Questions

Answer these questions in your own words. (2 pts each – 10 pts total)

Q1. Why does the server return a redirectTo value to the client instead of the client deciding where to send the user based on the role? What is the advantage?

Answer:
The server returns the redirectTo value because it is the one that checks the user's role during login. This makes the process more organized because the client only needs to follow the destination given by the server. It also keeps the role-based login logic in one place, making it easier to maintain.

Q2. Since the users array is stored in memory, what happens to all accounts when the server is stopped and restarted? How would you fix this problem in a real application?

Answer:
When the server is stopped or restarted, all accounts stored in the users array are lost because the data only exists in memory. To fix this in a real application, I would use a database such as MySQL or MongoDB to store user accounts permanently.

Q3. The login route returns the same error message — "Invalid email or password" — whether the email is unknown or the password is wrong. Why is this considered good practice?

Answer:
This is good practice because it does not tell attackers whether an email address is registered in the system. Using the same error message for both cases helps protect users' account information and makes it harder for someone to identify valid accounts.

Q4. The sanitizeUser function removes the password before sending the user object to the client. What could go wrong if the password were included in the JSON response?

Answer:
If the password were included in the JSON response, someone who could access the response might see the user's password. This could put the account at risk, especially if the user uses the same password on other websites. Removing the password helps protect sensitive information.

Q5. The admin dashboard checks sessionStorage.getItem('currentUser') before showing content. Is this a strong form of protection? Explain why or why not?

Answer:
No, it is not a strong form of protection because sessionStorage is stored in the browser and can be modified by the user. Someone could change the stored role and gain access to the dashboard interface. A

real application should verify the user's identity and permissions on the server for every protected request.

IV. Project Setup

Step 1: Create the project folder and install dependencies

```
mkdir role-login-lab
cd role-login-lab
npm init -y
npm install express
```

Step 2: Create the folder structure

```
role-login-lab/
├── server.js
├── package.json
└── public/
    ├── login.html
    ├── admin.html
    ├── staff.html
    ├── user.html
    └── style.css
```

Step 3: Verify installation

Check that node_modules folder and package.json exist.

V. Sample Code

5.1 server.js

```
const express = require('express');
const path = require('path');
const app = express();

// Middleware to parse JSON request bodies
app.use(express.json());

// Serve the public folder (login.html, dashboards, css)
app.use(express.static(path.join(__dirname, 'public')));

// ---------- In-memory "database" ----------
// Starts EMPTY. Populate it via POST /api/users from Postman.
let users = [];
let nextId = 1;

// ---------- Helpers ----------
const findUserById = (id) => users.find(u => u.id === id);

const findUserByEmail = (email) =>
  users.find(u => u.email.toLowerCase() ===
String(email).toLowerCase());
```

```
// Strip password before sending to the client
const sanitizeUser = (user) => {
  const { password, ...safe } = user;
  return safe;
};

// ---------- User CRUD Routes ----------

// GET /api/users - List all users
app.get('/api/users', (req, res) => {
  res.json(users.map(sanitizeUser));
});

// POST /api/users - Create a new user
app.post('/api/users', (req, res) => {
  const { name, email, password, role } = req.body;

  // Basic validation
  if (!name || !email || !password) {
    return res.status(400).json({ error: 'Name, email, and password are
required.' });
  }

  // Simple email format check
  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  if (!emailRegex.test(email)) {
    return res.status(400).json({ error: 'Invalid email format.' });
  }

  // Prevent duplicate accounts (case-insensitive)
  if (findUserByEmail(email)) {
    return res.status(409).json({ error: 'Email already registered.' });
  }

  // Normalize role: 'admin' | 'staff' | 'user'
  let normalizedRole = 'user';
  if (role === 'admin') normalizedRole = 'admin';
  else if (role === 'staff') normalizedRole = 'staff';

  const newUser = {
    id: nextId++,
    name,
    email,
    password, // plain text - DEV ONLY
    role: normalizedRole
  };

  users.push(newUser);
  res.status(201).json(sanitizeUser(newUser));
```

```
});

// GET /api/users/:id - Get user by ID
app.get('/api/users/:id', (req, res) => {
  const id = parseInt(req.params.id, 10);
  if (isNaN(id)) {
    return res.status(400).json({ error: 'Invalid user ID.' });
  }

  const user = findUserById(id);
  if (!user) {
    return res.status(404).json({ error: 'User not found.' });
  }

  res.json(sanitizeUser(user));
});

// PUT /api/users/:id - Update user
app.put('/api/users/:id', (req, res) => {
  const id = parseInt(req.params.id, 10);
  if (isNaN(id)) {
    return res.status(400).json({ error: 'Invalid user ID.' });
  }

  const { name, email, password, role } = req.body;

  if (!name || !email) {
    return res.status(400).json({ error: 'Name and email are required.'
});
  }

  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  if (!emailRegex.test(email)) {
    return res.status(400).json({ error: 'Invalid email format.' });
  }

  const user = findUserById(id);
  if (!user) {
    return res.status(404).json({ error: 'User not found.' });
  }

  const existing = findUserByEmail(email);
  if (existing && existing.id !== id) {
    return res.status(409).json({ error: 'Email already registered.' });
  }

  user.name = name;
  user.email = email;
  if (password) user.password = password; // only change if provided
```

```
  if (role === 'admin') user.role = 'admin';
  else if (role === 'staff') user.role = 'staff';
  else if (role === 'user') user.role = 'user';

  res.json(sanitizeUser(user));
});

// DELETE /api/users/:id - Delete user
app.delete('/api/users/:id', (req, res) => {
  const id = parseInt(req.params.id, 10);
  if (isNaN(id)) {
    return res.status(400).json({ error: 'Invalid user ID.' });
  }

  const index = users.findIndex(u => u.id === id);
  if (index === -1) {
    return res.status(404).json({ error: 'User not found.' });
  }

  users.splice(index, 1);
  res.status(204).send();
});

// ---------- Login Route ----------
app.post('/api/login', (req, res) => {
  const { email, password } = req.body;

  if (!email || !password) {
    return res.status(400).json({ error: 'Email and password are
required.' });
  }

  const user = findUserByEmail(email);

  // Same generic message for both cases (don't leak which one failed)
  if (!user || user.password !== password) {
    return res.status(401).json({ error: 'Invalid email or password.'
});
  }

  // Server decides where the client should go, based on role
  let redirectTo = '/user.html';
  if (user.role === 'admin') redirectTo = '/admin.html';
  else if (user.role === 'staff') redirectTo = '/staff.html';

  res.json({
    message: 'Login successful',
    user: sanitizeUser(user),
```

```
    redirectTo
  });
});

// ---------- Start Server ----------
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
  console.log(`Login page: http://localhost:${PORT}/login.html`);
});
```

5.2 public/login.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0"
/>
  <title>Sign In - Role-Based Lab</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body class="centered">
  <div class="card">
    <h1>Sign In</h1>
    <p class="muted">Use an account created via Postman</p>

    <form id="loginForm" autocomplete="off">
      <label>
        Email
        <input type="email" id="email" placeholder="you@lab.com"
required />
      </label>
      <label>
        Password
        <input type="password" id="password" placeholder="......"
required />
      </label>
      <button type="submit" class="btn primary">Login</button>
    </form>

    <p id="msg" class="msg"></p>

    <div class="hint">
      <strong>No accounts yet?</strong>
      <div>Create one in Postman first:</div>
      <div><code>POST http://localhost:3000/api/users</code></div>
      <div>Body: <code>{ "name", "email", "password", "role"
}</code></div>
```

```
      <div>Roles: <code>admin</code> · <code>staff</code> ·
<code>user</code></div>
    </div>
  </div>

  <script>
    const form = document.getElementById('loginForm');
    const msg = document.getElementById('msg');

    form.addEventListener('submit', async (e) => {
      e.preventDefault();
      msg.textContent = '';
      msg.className = 'msg';

      const email = document.getElementById('email').value.trim();
      const password = document.getElementById('password').value;

      // Client-side validation
      if (!email.includes('@')) {
        msg.textContent = 'Please enter a valid email address.';
        msg.classList.add('error');
        return;
      }
      if (password.length < 6) {
        msg.textContent = 'Password must be at least 6 characters.';
        msg.classList.add('error');
        return;
      }

      try {
        const res = await fetch('/api/login', {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ email, password })
        });

        const data = await res.json();

        if (!res.ok) {
          msg.textContent = data.error || 'Login failed.';
          msg.classList.add('error');
          return;
        }

        // Save the logged-in user for the dashboard pages
        sessionStorage.setItem('currentUser',
JSON.stringify(data.user));

        // Follow the server's redirect decision
```

```
        window.location.href = data.redirectTo;
      } catch (err) {
        msg.textContent = 'Cannot reach the server. Is it running?';
        msg.classList.add('error');
      }
    });
  </script>
</body>
</html>
```

5.3 public/admin.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0"
/>
  <title>Admin Dashboard</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body>
  <div class="container">
    <header class="topbar">
      <h1>Admin Dashboard</h1>
      <button id="logoutBtn" class="btn danger">Logout</button>
    </header>

    <section class="card">
      <h2>Welcome, <span id="adminName">...</span></h2>
      <p class="muted">Signed in as <strong id="adminEmail"></strong>
(role: admin)</p>
    </section>

    <section class="card">
      <h2>Registered Users</h2>
      <p class="muted">These are the accounts stored in the server's
memory.</p>
      <table>
        <thead>
          <tr><th>ID</th><th>Name</th><th>Email</th><th>Role</th></tr>
        </thead>
        <tbody id="userTableBody"></tbody>
      </table>
    </section>
  </div>

  <script>
```

```
    const currentUser =
JSON.parse(sessionStorage.getItem('currentUser'));

    // Simple client-side guard
    if (!currentUser || currentUser.role !== 'admin') {
      window.location.href = '/login.html';
    } else {
      document.getElementById('adminName').textContent =
currentUser.name;
      document.getElementById('adminEmail').textContent =
currentUser.email;
    }

    document.getElementById('logoutBtn').addEventListener('click', () =>
{
      sessionStorage.removeItem('currentUser');
      window.location.href = '/login.html';
    });

    async function loadUsers() {
      try {
        const res = await fetch('/api/users');
        const list = await res.json();
        const tbody = document.getElementById('userTableBody');

        if (!list.length) {
          tbody.innerHTML = '<tr><td colspan="4">No users yet. Create
some in Postman.</td></tr>';
          return;
        }

        tbody.innerHTML = list.map(u => `
          <tr>
            <td>${u.id}</td>
            <td>${u.name}</td>
            <td>${u.email}</td>
            <td><span class="badge ${u.role}">${u.role}</span></td>
          </tr>
        `).join('');
      } catch {
        document.getElementById('userTableBody').innerHTML =
          '<tr><td colspan="4">Failed to load users.</td></tr>';
      }
    }

    if (currentUser && currentUser.role === 'admin') loadUsers();
  </script>
</body>
</html>
```

5.4 public/staff.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0"
/>
  <title>Staff Dashboard</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body>
  <div class="container">
    <header class="topbar">
      <h1>Staff Dashboard</h1>
      <button id="logoutBtn" class="btn danger">Logout</button>
    </header>

    <section class="card">
      <h2>Welcome, <span id="staffName">...</span></h2>
      <p class="muted">Signed in as <strong
id="staffEmail"></strong></p>
    </section>

    <section class="card">
      <h2>Staff Tools</h2>
      <ul>
        <li>Review submitted records</li>
        <li>Update content and announcements</li>
        <li>Reply to user inquiries</li>
      </ul>
      <p class="muted">Staff members cannot manage user accounts.</p>
    </section>
  </div>

  <script>
    const currentUser =
JSON.parse(sessionStorage.getItem('currentUser'));

    if (!currentUser || currentUser.role !== 'staff') {
      window.location.href = '/login.html';
    } else {
      document.getElementById('staffName').textContent =
currentUser.name;
      document.getElementById('staffEmail').textContent =
currentUser.email;
    }
```

```
    document.getElementById('logoutBtn').addEventListener('click', () =>
{
      sessionStorage.removeItem('currentUser');
      window.location.href = '/login.html';
    });
  </script>
</body>
</html>
```

5.5 public/user.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0"
/>
  <title>User Dashboard</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body>
  <div class="container">
    <header class="topbar">
      <h1>User Dashboard</h1>
      <button id="logoutBtn" class="btn danger">Logout</button>
    </header>

    <section class="card">
      <h2>Welcome, <span id="userName">...</span></h2>
      <p class="muted">Signed in as <strong id="userEmail"></strong></p>
    </section>
  </div>

  <script>
    const currentUser =
JSON.parse(sessionStorage.getItem('currentUser'));

    if (!currentUser || currentUser.role !== 'user') {
      window.location.href = '/login.html';
    } else {
      document.getElementById('userName').textContent =
currentUser.name;
      document.getElementById('userEmail').textContent =
currentUser.email;
    }

    document.getElementById('logoutBtn').addEventListener('click', () =>
{
      sessionStorage.removeItem('currentUser');
```

```
      window.location.href = '/login.html';
    });
  </script>
</body>
</html>
```

5.6 public/style.css

```
* {
  box-sizing: border-box;
  font-family: system-ui, -apple-system, "Segoe UI", Roboto, sans-serif;
}

body {
  margin: 0;
  background: #f4f7fb;
  color: #1e293b;
}

.centered {
  min-height: 100vh;
  display: grid;
  place-items: center;
  padding: 1rem;
}

.container {
  max-width: 960px;
  margin: 0 auto;
  padding: 1.5rem;
}

.card {
  background: #fff;
  border-radius: 14px;
  padding: 1.5rem;
  box-shadow: 0 10px 25px rgb(0, 0, 0, .06);
  margin-bottom: 1.25rem;
}

h1 { margin: 0 0 .25rem; font-size: 1.6rem; }
h2 { margin: 0 0 .5rem; font-size: 1.2rem; }
.muted { color: #64748b; margin: 0 0 1rem; }

label {
  display: block;
  margin-bottom: .85rem;
  font-weight: 600;
  font-size: .9rem;
```

```
}

input {
  width: 100%;
  padding: .7rem .85rem;
  border: 1px solid #cbd5e1;
  border-radius: 10px;
  margin-top: .35rem;
  font-size: 1rem;
}

input:focus {
  outline: 2px solid #3b82f6;
  border-color: transparent;
}

.btn {
  border: 0;
  border-radius: 10px;
  padding: .7rem 1rem;
  font-weight: 700;
  cursor: pointer;
  font-size: .95rem;
}

.primary { background: #2563eb; color: #fff; width: 100%; }
.primary:hover { background: #1d4ed8; }

.danger { background: #ef4444; color: #fff; }
.danger:hover { background: #dc2626; }

.msg { min-height: 1.2rem; font-size: .9rem; margin: .75rem 0 0; }
.error { color: #dc2626; }

.hint {
  margin-top: 1rem;
  padding: .75rem;
  background: #f8fafc;
  border: 1px dashed #cbd5e1;
  border-radius: 10px;
  font-size: .85rem;
  color: #475569;
}

.hint code {
  background: #e2e8f0;
  padding: .1rem .3rem;
  border-radius: 5px;
}
```

```
.topbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 1.25rem;
}

table { width: 100%; border-collapse: collapse; }

th, td {
  text-align: left;
  padding: .75rem;
  border-bottom: 1px solid #e2e8f0;
}

th {
  background: #f1f5f9;
  font-size: .85rem;
  text-transform: uppercase;
  letter-spacing: .03em;
}

.badge {
  padding: .2rem .5rem;
  border-radius: 999px;
  font-size: .75rem;
  font-weight: 700;
  text-transform: uppercase;
}

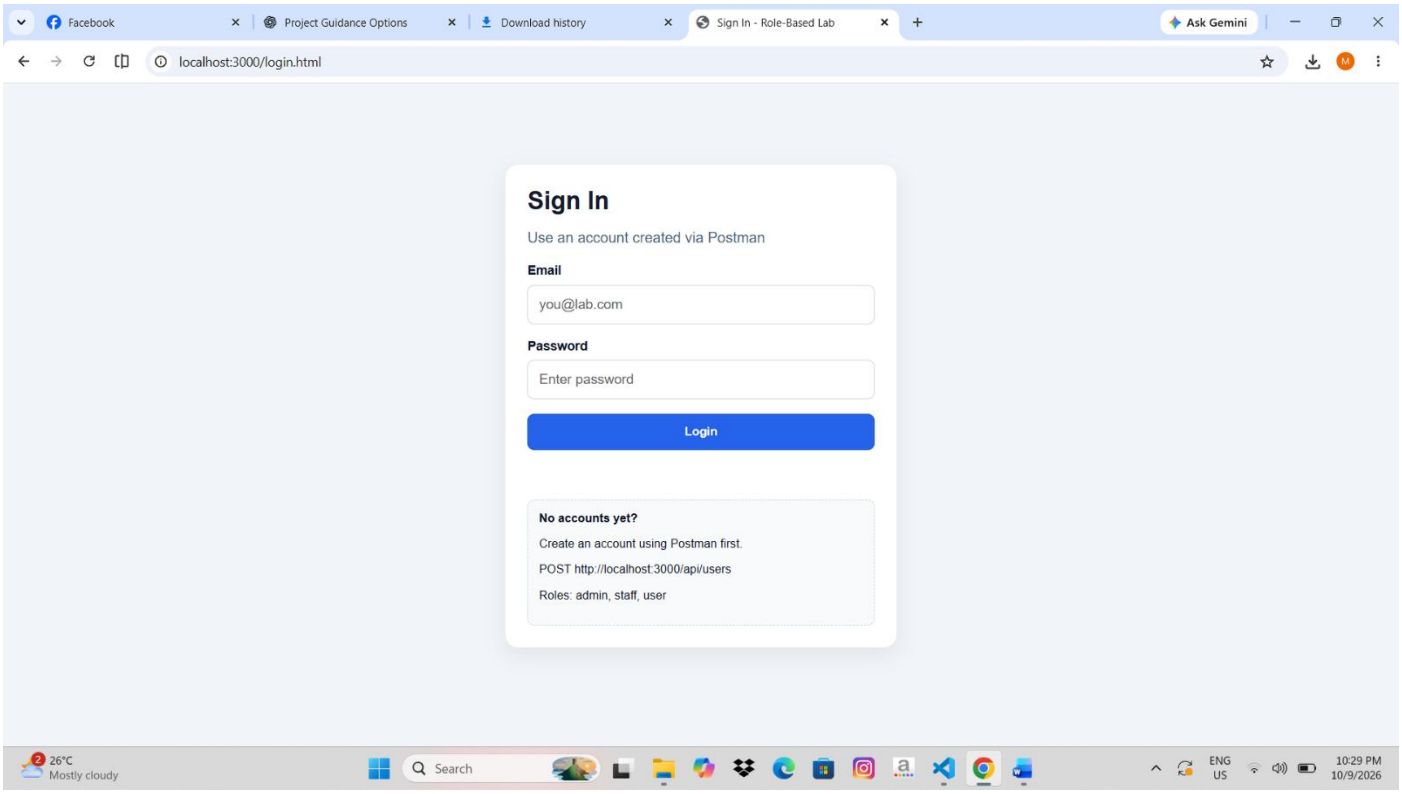
.badge.admin { background: #dbeafe; color: #1e40af; }
.badge.staff { background: #fef3c7; color: #92400e; }
.badge.user { background: #dcfce7; color: #166534; }
```

VI. Laboratory Tasks

Task 1 — Run the Server and Verify Static Files (5 pts)

1. Run `node server.js`.
2. Confirm the console prints both URLs.
3. Open `http://localhost:3000/login.html` in your browser.
4. **Expected:** The login form appears with the "No accounts yet?" hint box.

Deliverable: Screenshot of the login page.

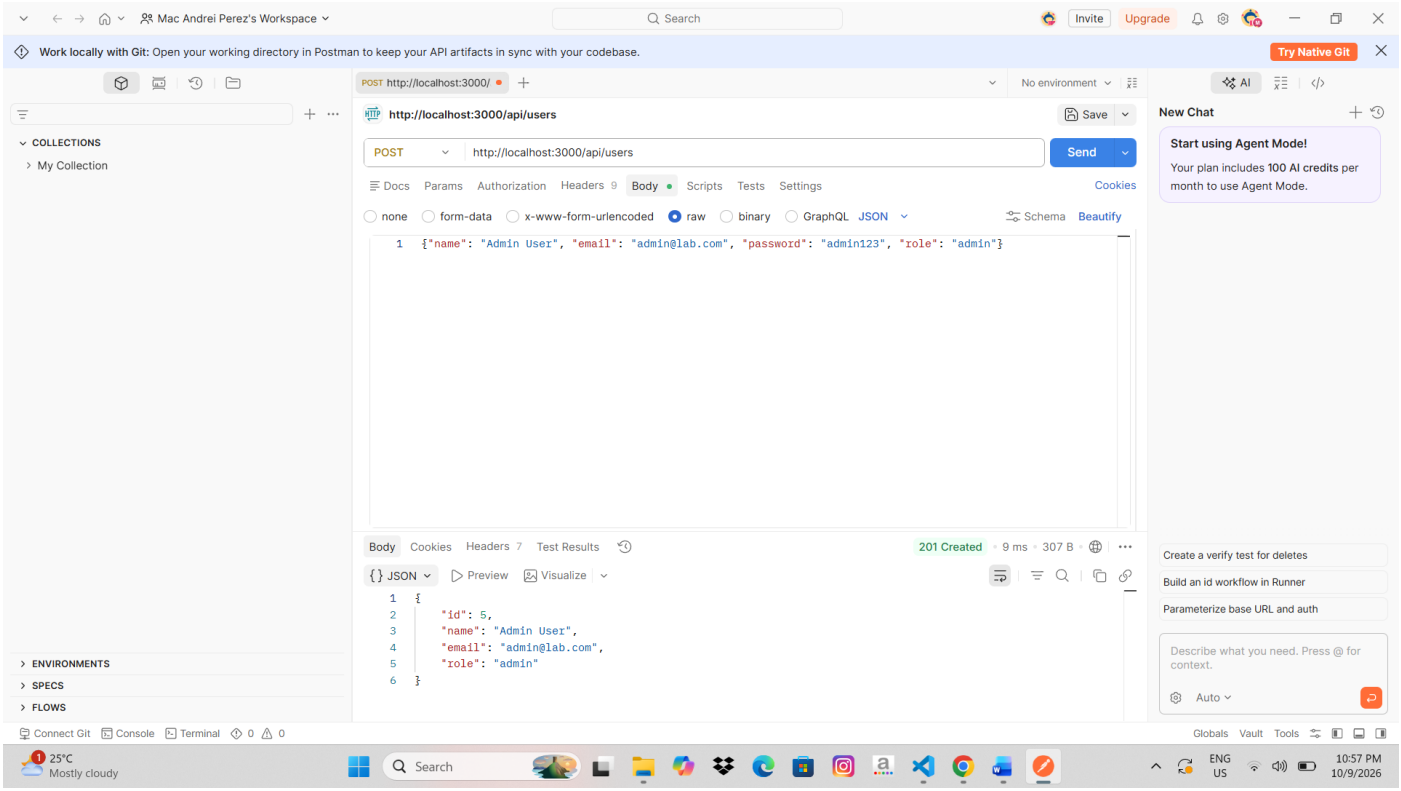


Task 2 — Create Users via Postman or Talend Api (20 pts)

Base URL: `http://localhost:3000`

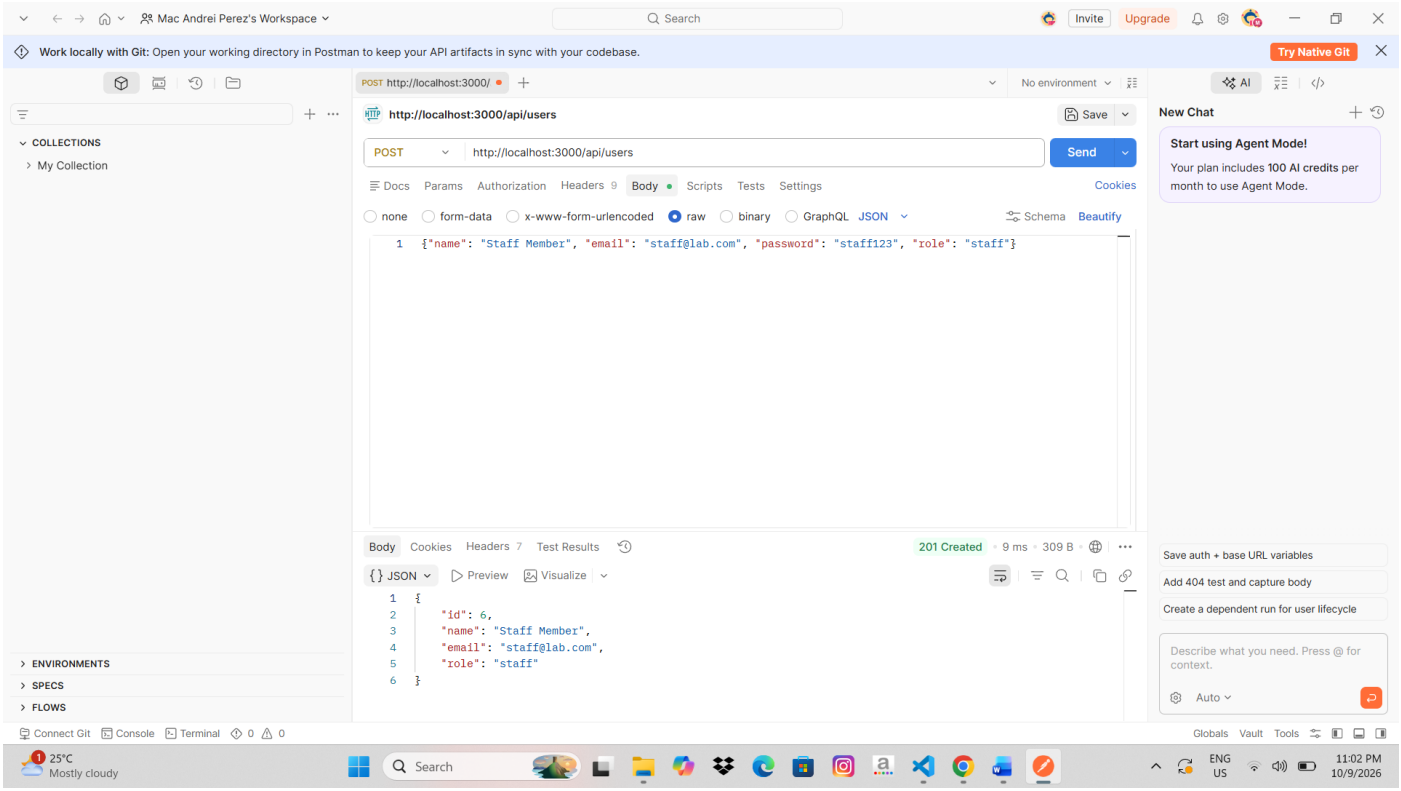
2a — Create an admin user

Field	Value
Method	POST
URL	http://localhost:3000/api/users
Headers	Content-Type: application/json
Body	{"name": "Admin User", "email": "admin@lab.com", "password": "admin123", "role": "admin"}
Expected Status	201
Expected Response	User object without password



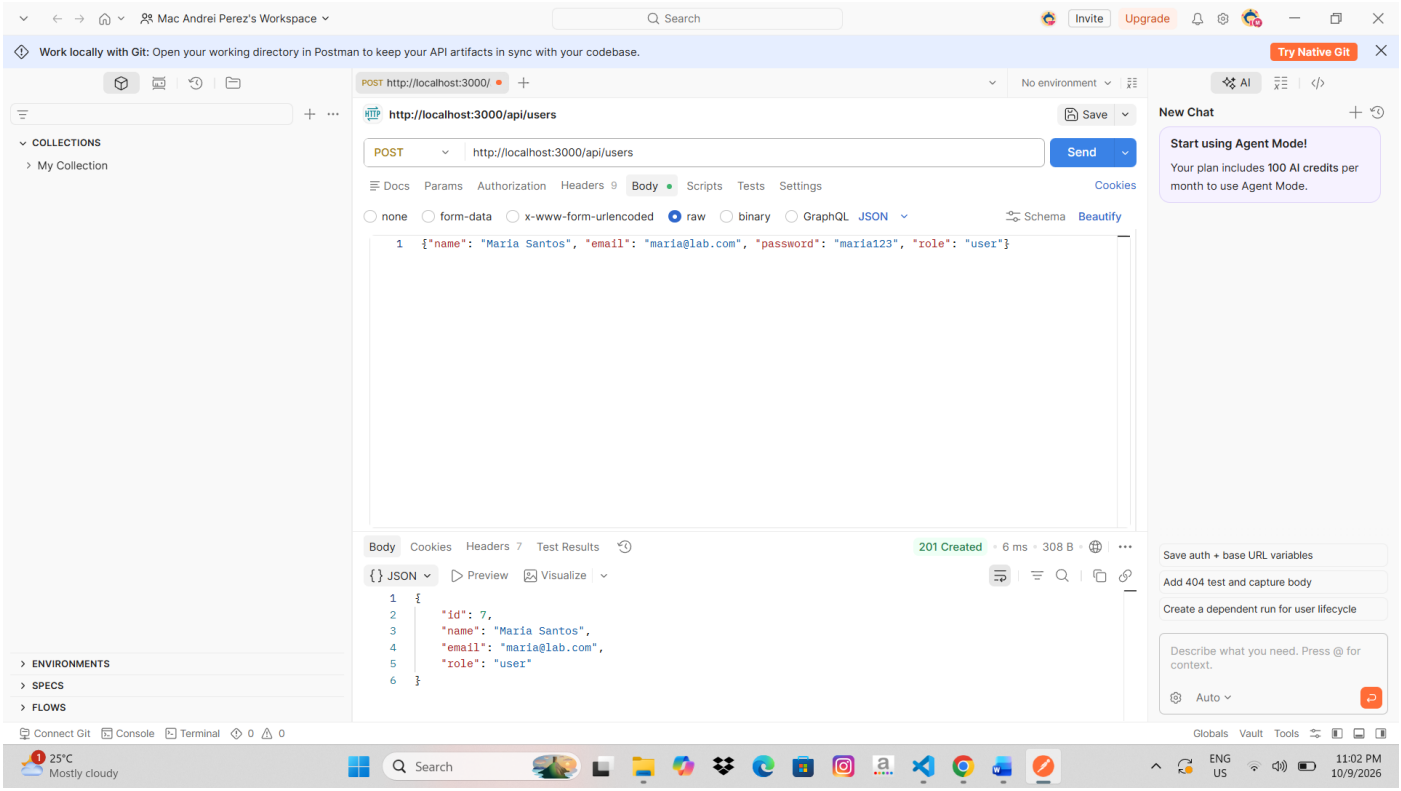
2b — Create a staff user

Field	Value
Method	POST
URL	http://localhost:3000/api/users
Body	{"name": "Staff Member", "email": "staff@lab.com", "password": "staff123", "role": "staff"}
Expected Status	201



2c — Create a normal user

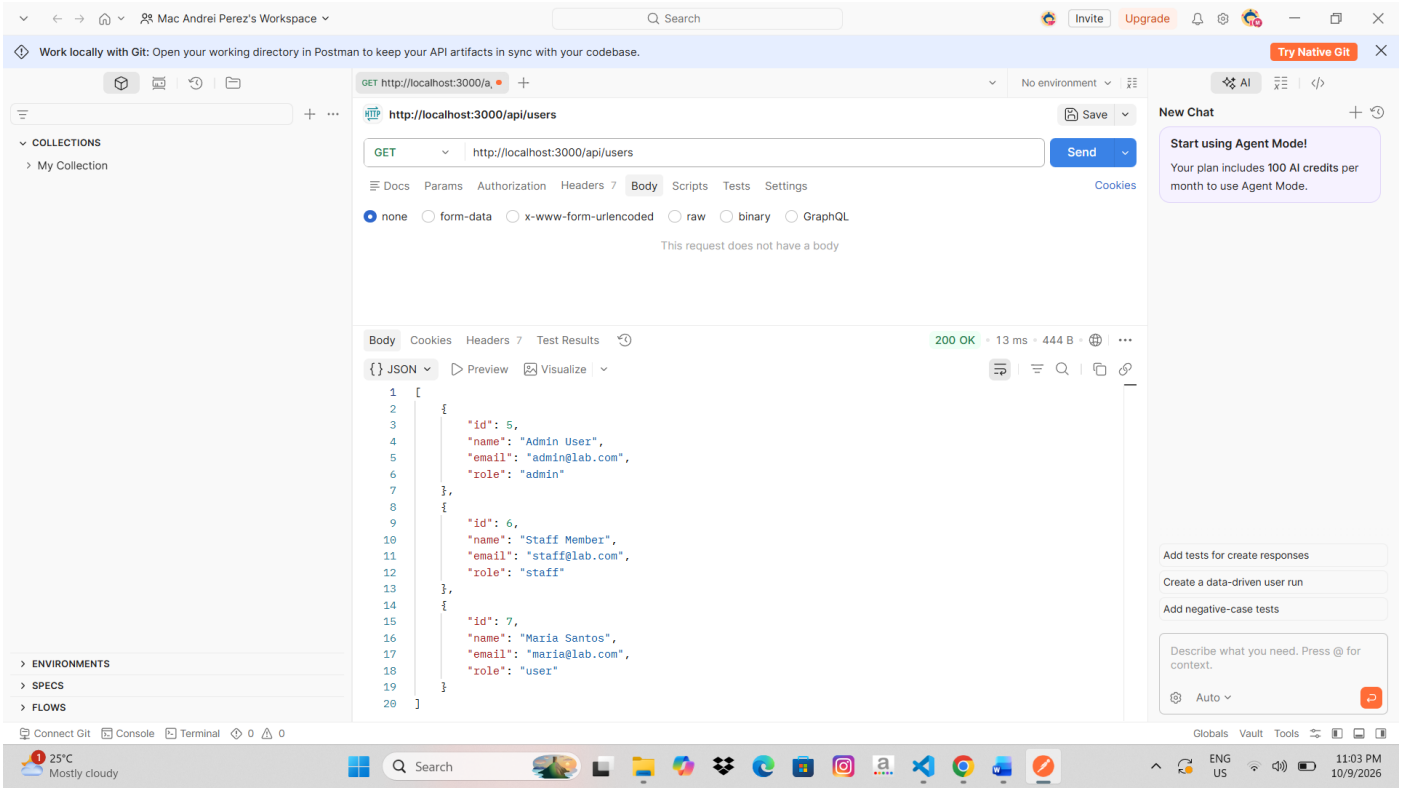
Field	Value
Method	POST
URL	http://localhost:3000/api/users
Body	{"name": "Maria Santos", "email": "maria@lab.com", "password": "maria123", "role": "user"}
Expected Status	201



2d — List all users

Field	Value
Method	GET
URL	http://localhost:3000/api/users
Expected Status	200
Expected Response	Array of all users without passwords

Deliverable: Screenshots of all four requests. The list must show all users without passwords.

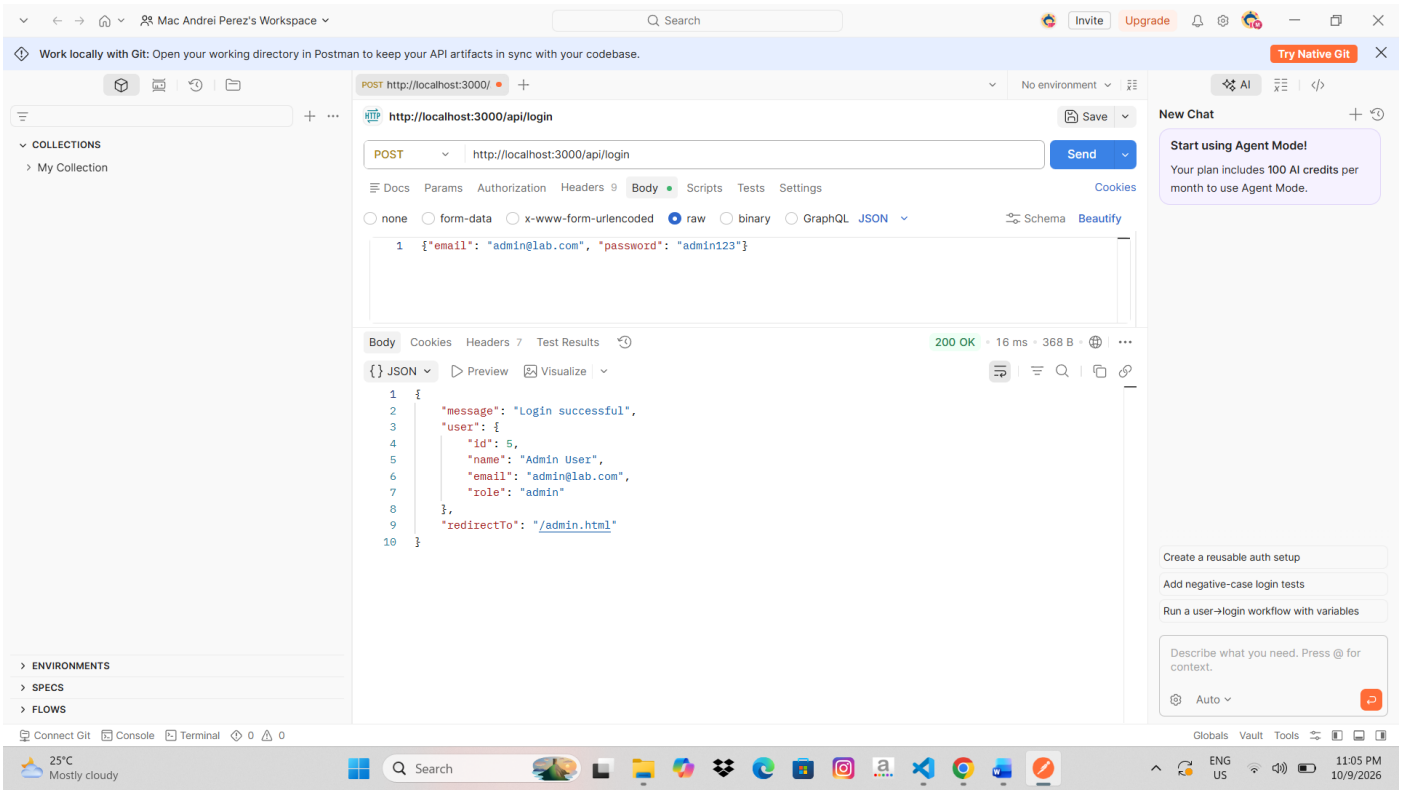


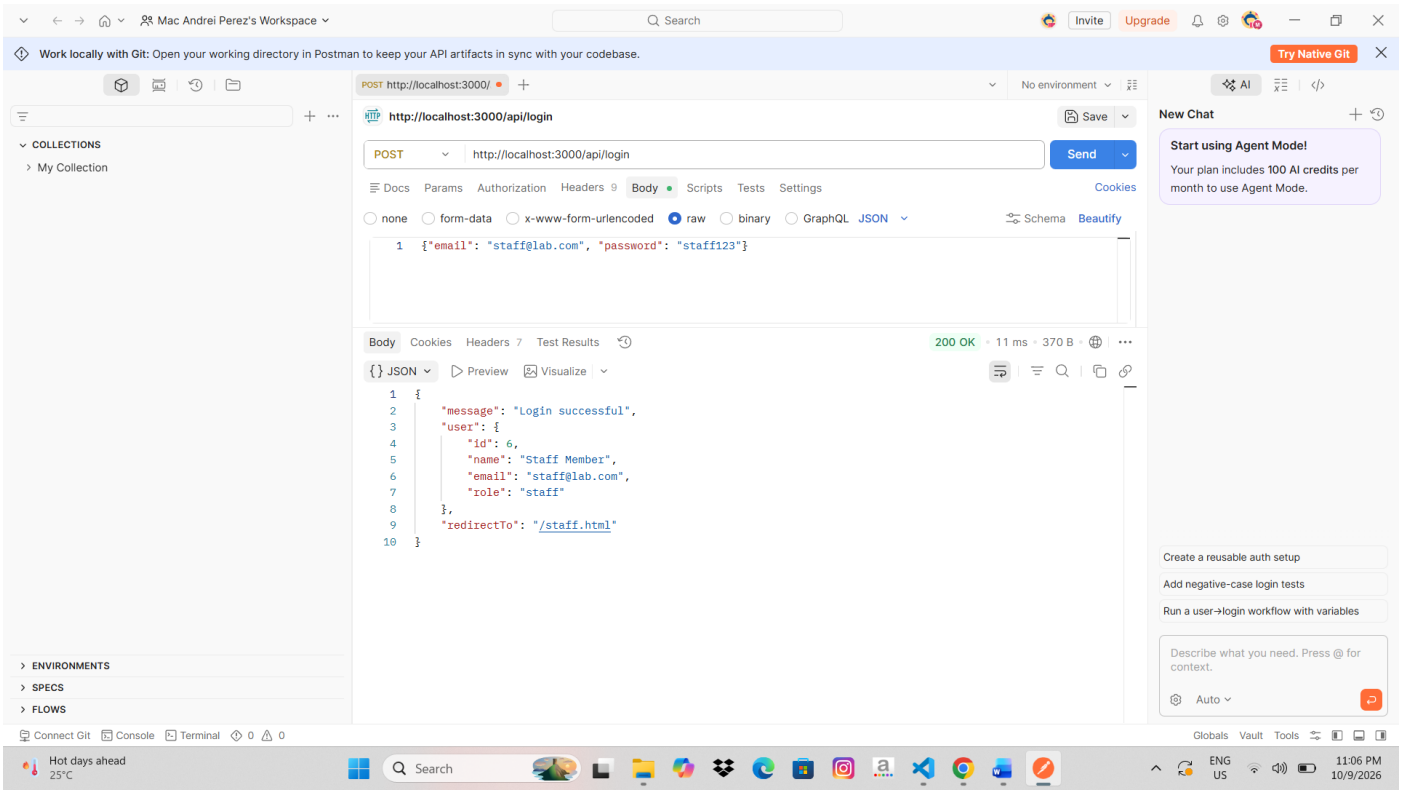
Task 3 — Test the Login API in Postman (10 pts)

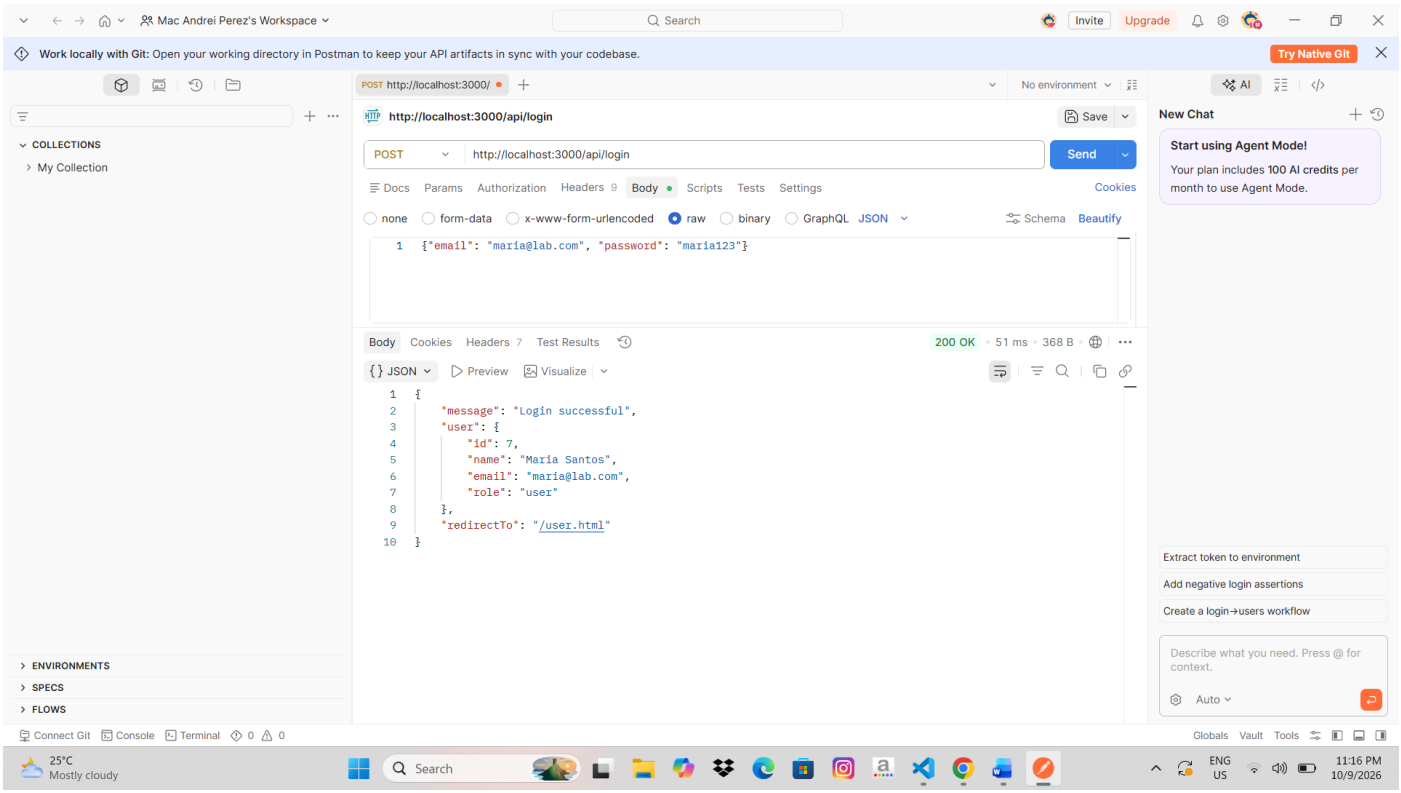
Test the following scenarios and record the actual status and response:

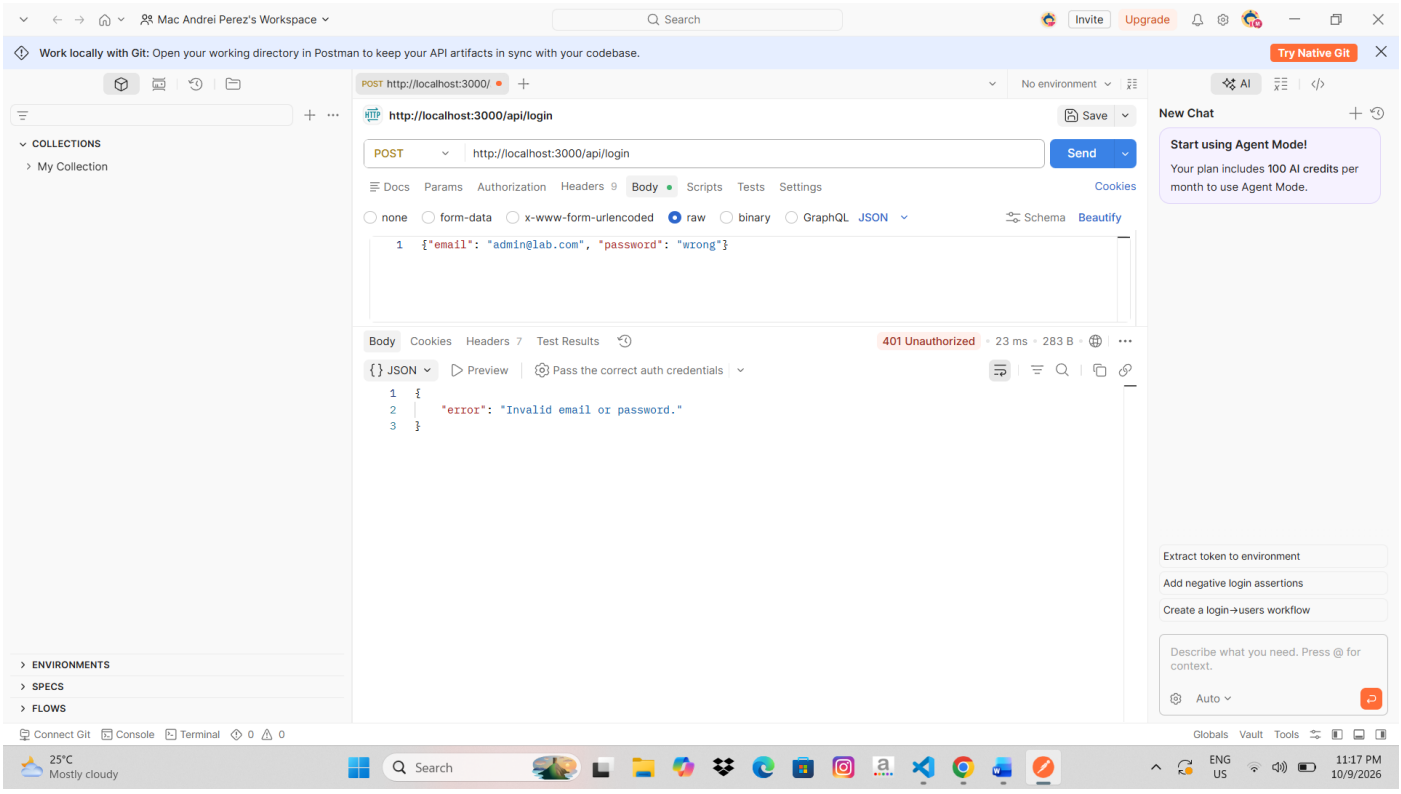
#	Body	Expected Status	Expected Response
1	{"email": "admin@lab.com", "password": "admin123"}	200	redirectTo: /admin.html
2	{"email": "staff@lab.com", "password": "staff123"}	200	redirectTo: /staff.html
3	{"email": "maria@lab.com", "password": "maria123"}	200	redirectTo: /user.html
4	{"email": "admin@lab.com", "password": "wrong"}	401	{"error": "Invalid email or password."}
5	{"email": "ghost@lab.com", "password": "admin123"}	401	{"error": "Invalid email or password."}
6	{"email": "admin@lab.com"}	400	{"error": "Email and password are required."}

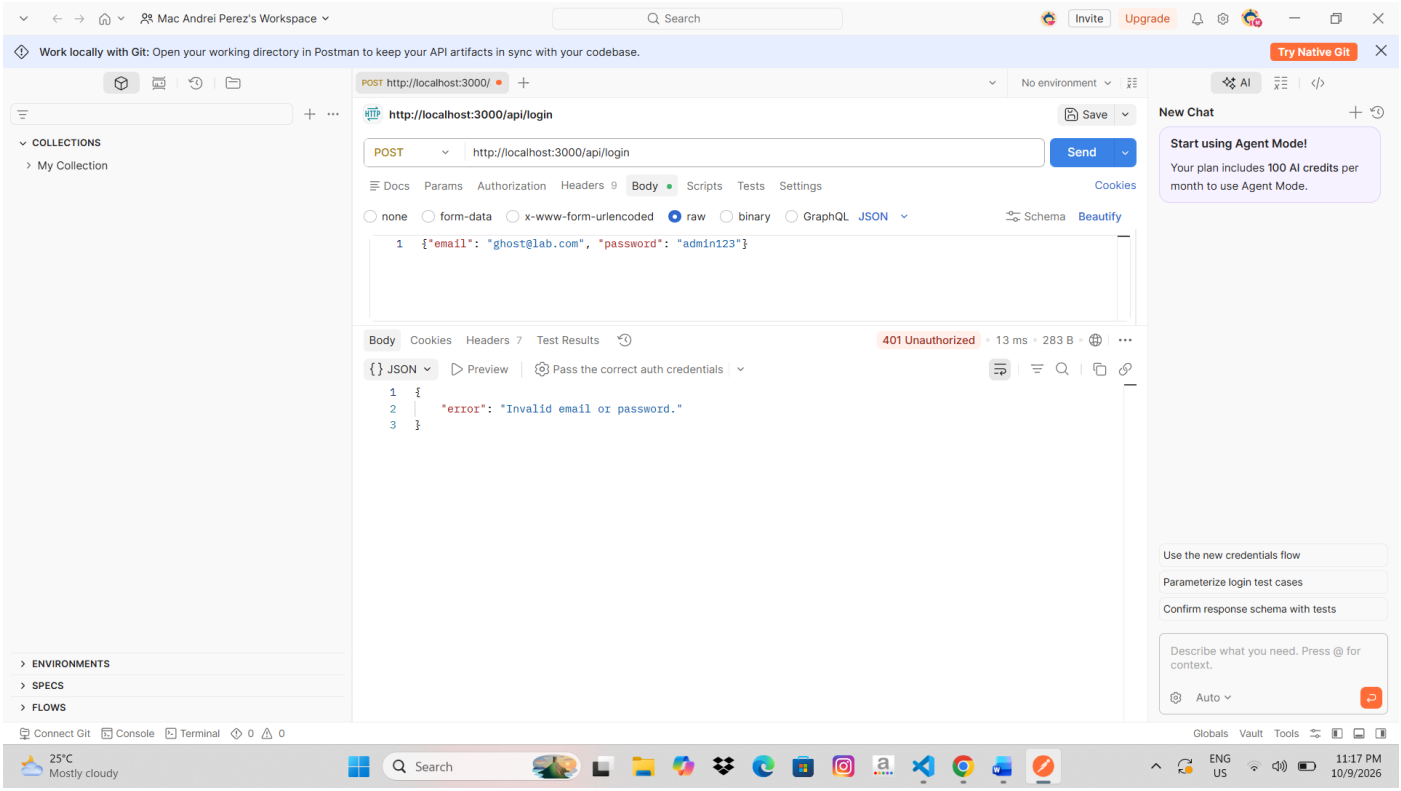
Deliverable: A table with actual status and response for each scenario.

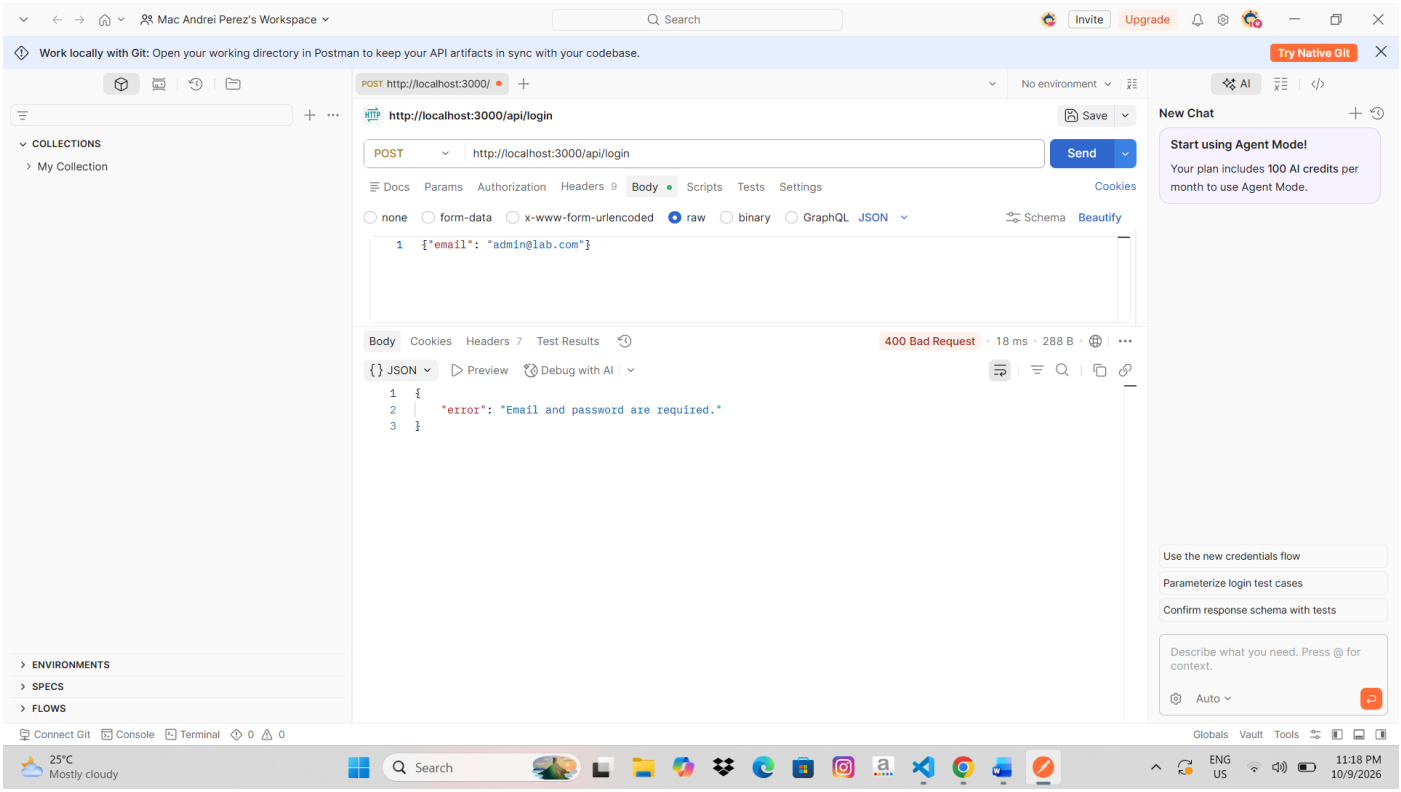





Mac Andrei Perez's Workspace
Search
Invite
Upgrade
Work locally with Git: Open your working directory in Postman to keep your API artifacts in sync with your codebase.
Try Native Git
POST http://localhost:3000/
No environment
http://localhost:3000/api/login
Save
COLLECTIONS
My Collection
POST
http://localhost:3000/api/login
Send
Docs Params Authorization Headers 9 Body Scripts Tests Settings
Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
1 {"email": "maria@lab.com", "password": "maria123"}
Body Cookies Headers 7 Test Results
200 OK 51 ms 368 B
{} JSON Preview Visualize
1 {
2 "message": "Login successful",
3 "user": {
4 "id": 7,
5 "name": "Maria Santos",
6 "email": "maria@lab.com",
7 "role": "user"
8 },
9 "redirectTo": "/user.html"
10 }
AI
New Chat
Start using Agent Mode!
Your plan includes 100 AI credits per month to use Agent Mode.
Extract token to environment
Add negative login assertions
Create a login→users workflow
Describe what you need. Press @ for context.
Auto
ENVIRONMENTS
SPECS
FLOWS
Connect Git Console Terminal 0 0
Globals Vault Tools
25°C
Mostly cloudy
Search
ENG
US
11:16 PM
10/9/2026


Mac Andrei Perez's Workspace
Search
Invite
Upgrade
Work locally with Git: Open your working directory in Postman to keep your API artifacts in sync with your codebase.
Try Native Git
POST http://localhost:3000/
No environment
http://localhost:3000/api/login
Save
COLLECTIONS
My Collection
POST
http://localhost:3000/api/login
Send
Docs Params Authorization Headers 9 Body Scripts Tests Settings
Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
1 {"email": "admin@lab.com", "password": "wrong"}
Body Cookies Headers 7 Test Results
401 Unauthorized 23 ms 283 B
{} JSON Preview Pass the correct auth credentials
1 {
2 "error": "Invalid email or password."
3 }
AI
New Chat
Start using Agent Mode!
Your plan includes 100 AI credits per month to use Agent Mode.
Extract token to environment
Add negative login assertions
Create a login→users workflow
Describe what you need. Press @ for context.
Auto
ENVIRONMENTS
SPECS
FLOWS
Connect Git Console Terminal 0 0
Globals Vault Tools
25°C
Mostly cloudy
Search
ENG
US
11:17 PM
10/9/2026

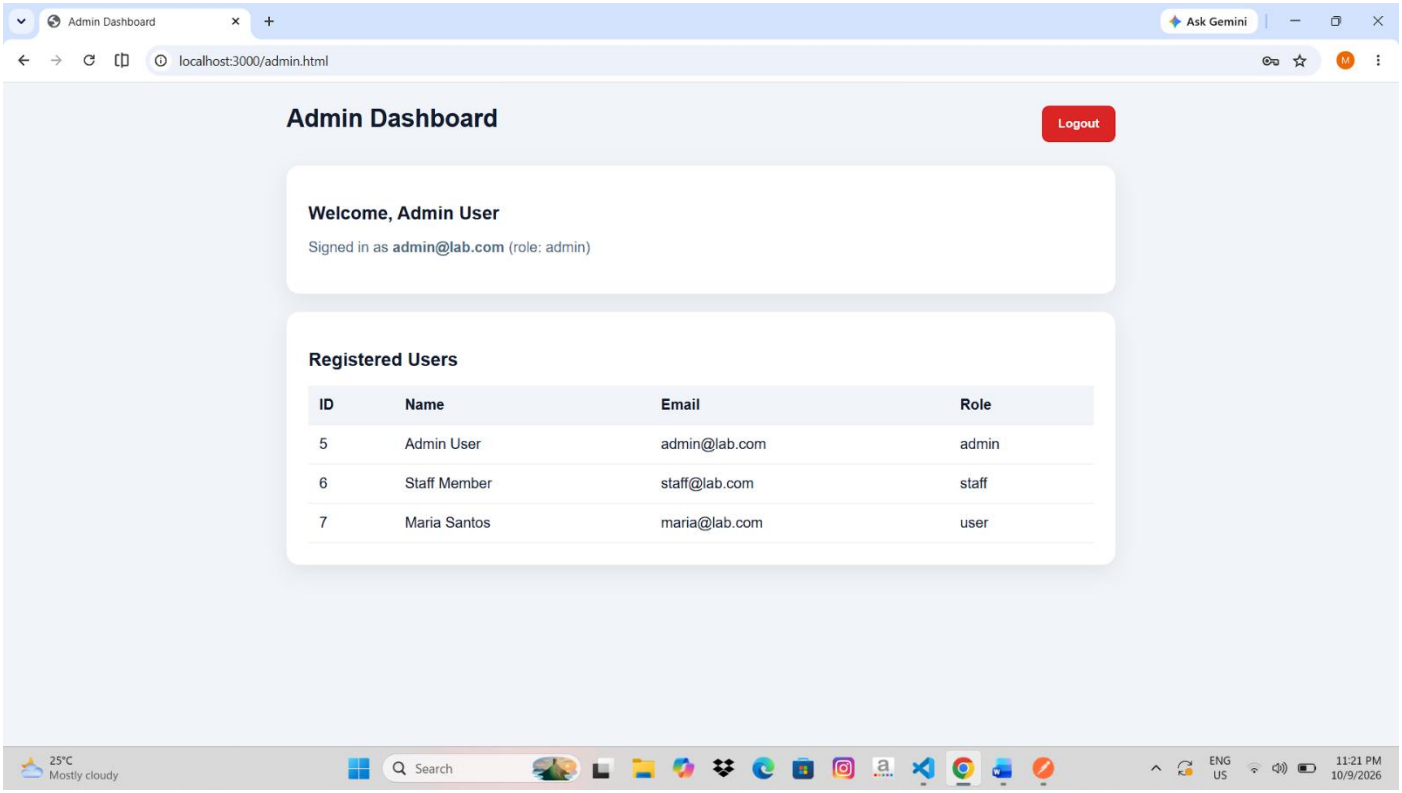




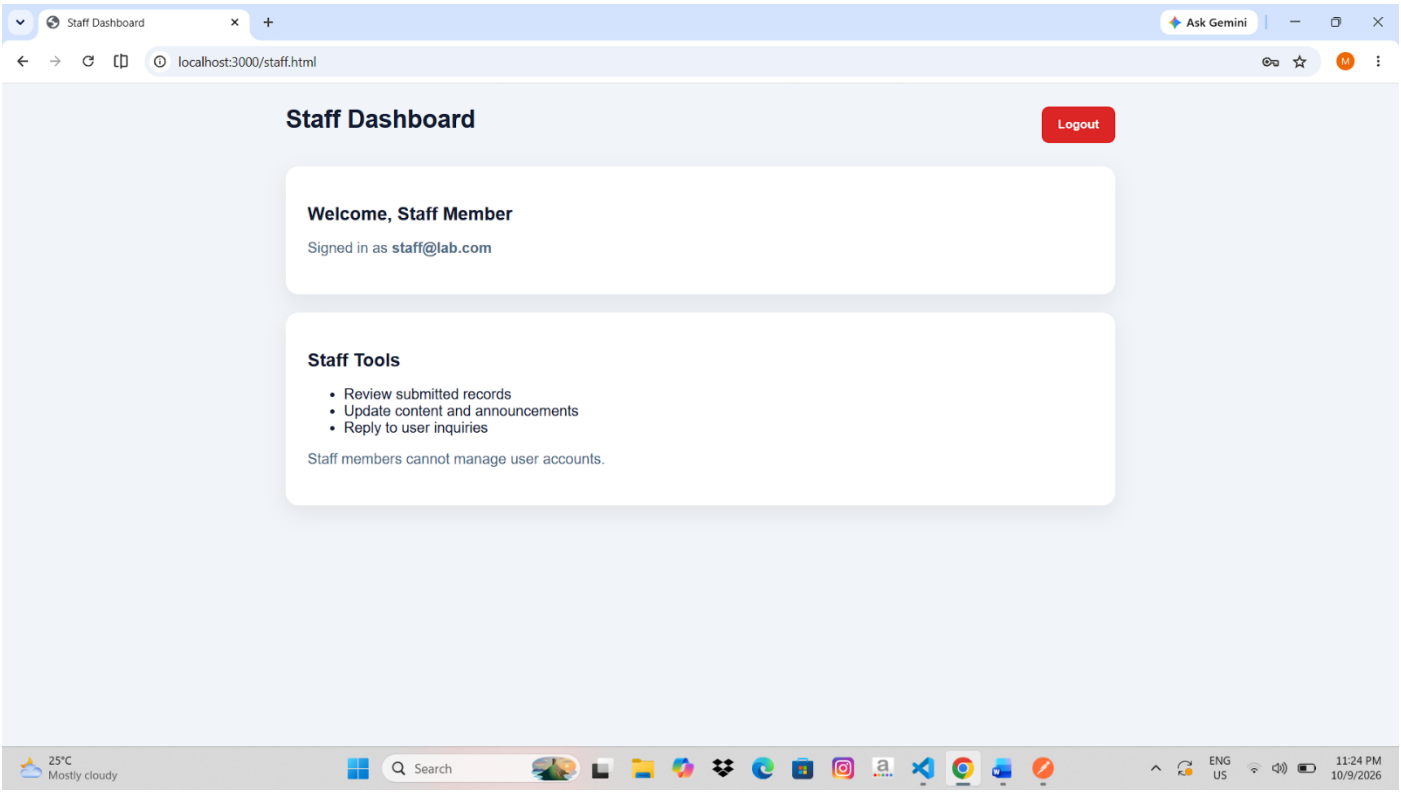
Task 4 — Test the Login Page in the Browser (10 pts)

1. Open http://localhost:3000/login.html.

2. Log in as **admin** — expect redirect to /admin.html with the user table.

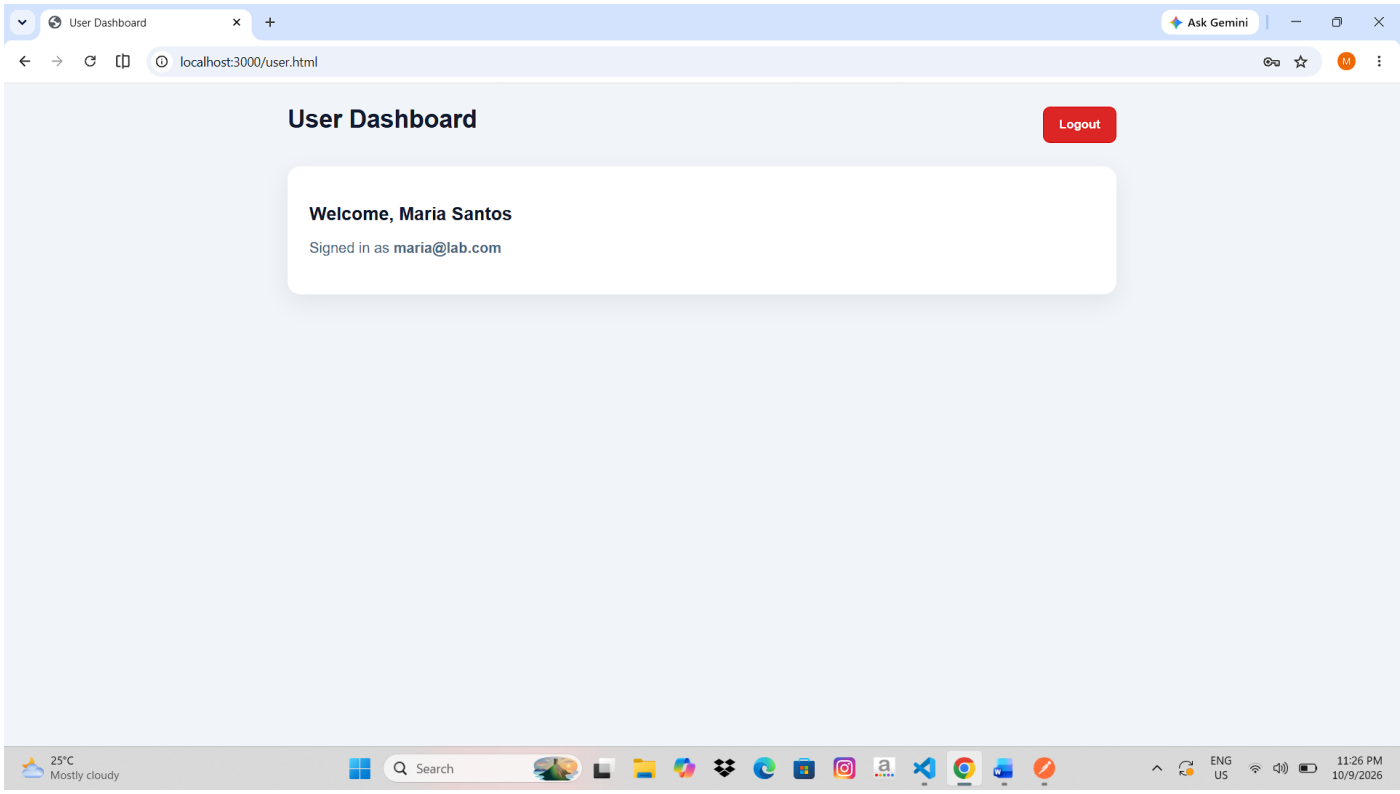


3. Logout.
4. Log in as **staff** — expect redirect to /staff.html.

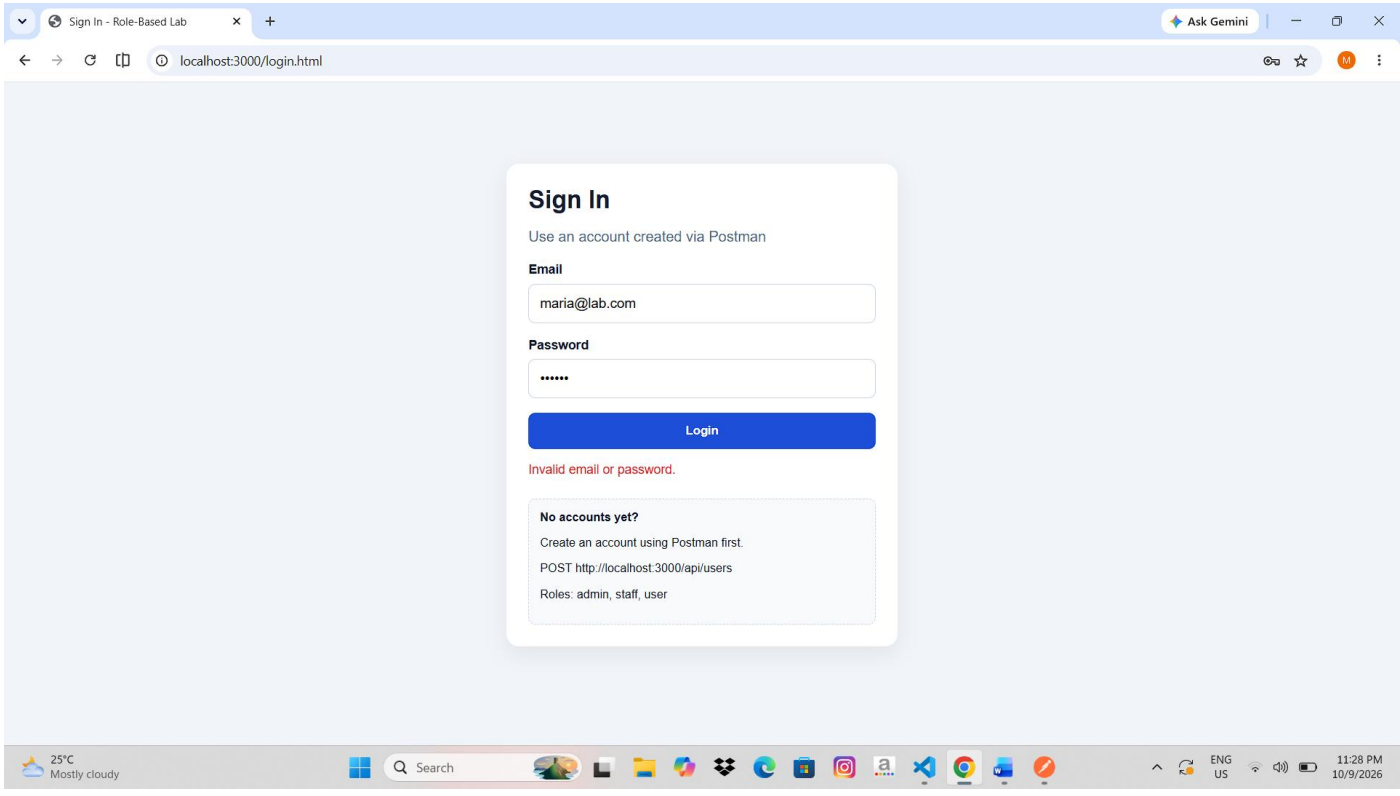


5. Logout.

6. Log in as **user** — expect redirect to /user.html.



7. Logout.
8. Log in with a **wrong password** — expect red error message.

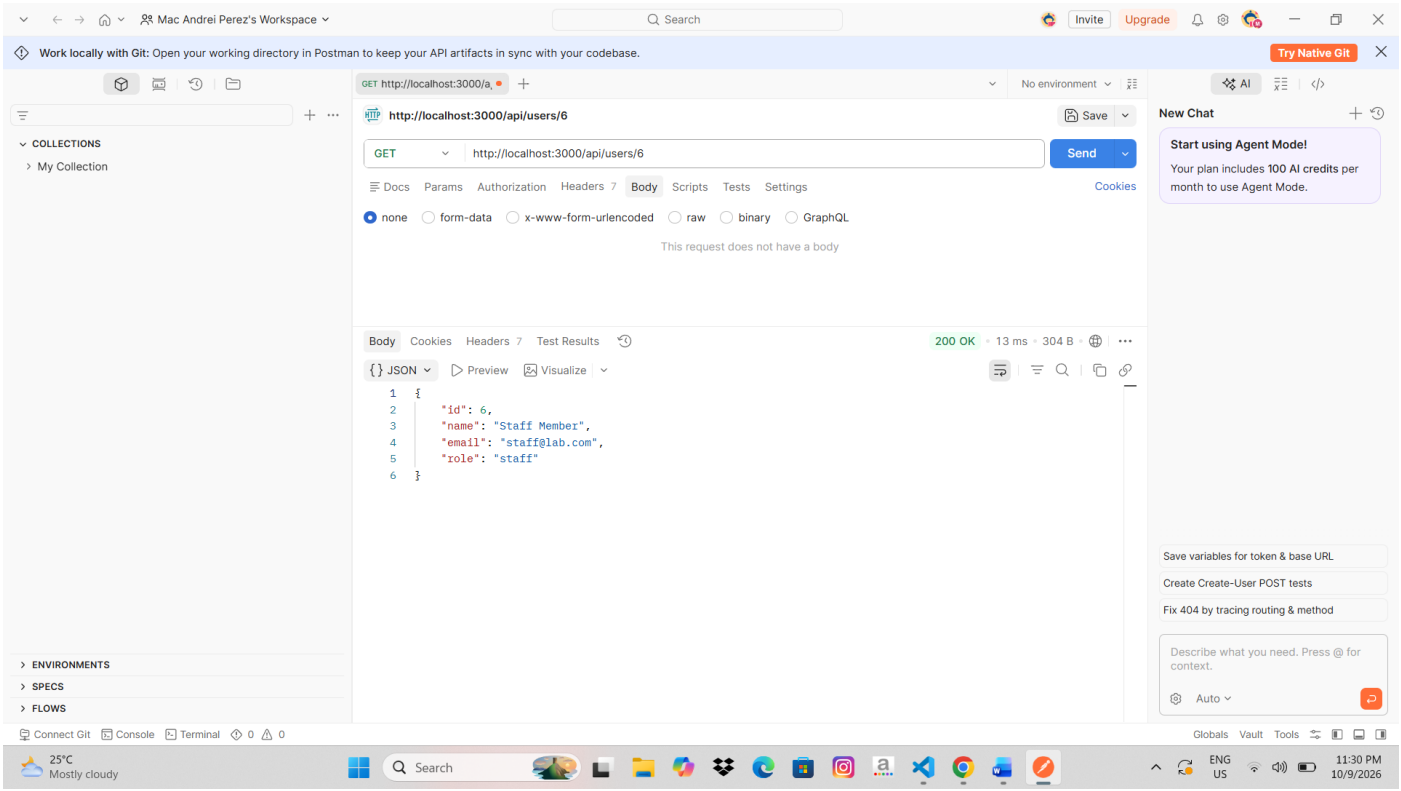


Deliverable: Screenshots of steps 2, 4, 6, and 8.

Task 5 — Test CRUD Operations in Postman (20 pts)

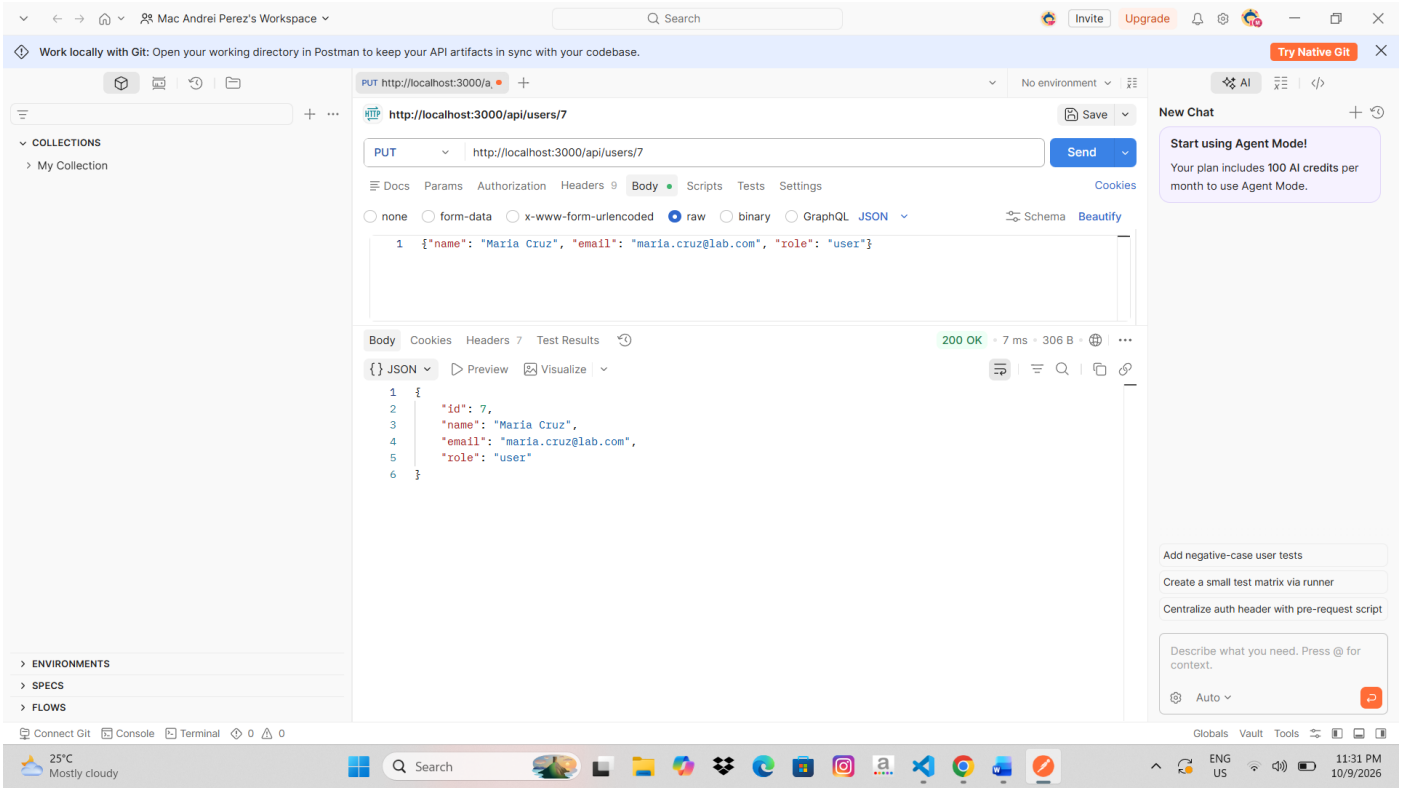
5a — GET single user

Field	Value
Method	GET
URL	http://localhost:3000/api/users/1
Expected Status	200
Expected Response	User with id=1, no password



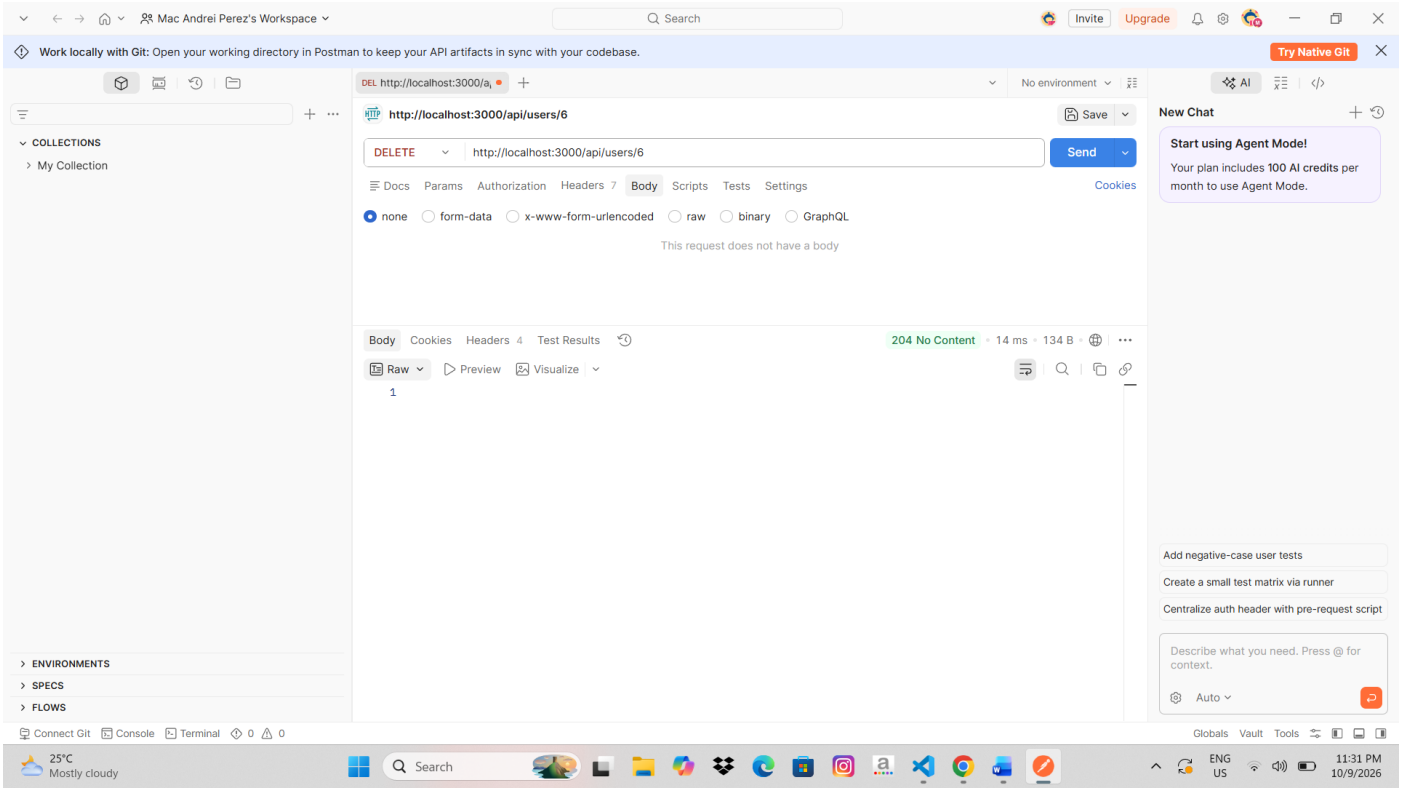
5b — PUT update user

Field	Value
Method	PUT
URL	http://localhost:3000/api/users/3
Body	{"name": "Maria Cruz", "email": "maria.cruz@lab.com", "role": "user"}
Expected Status	200
Expected Response	Updated user object


Mac Andrei Perez's Workspace
Search
Invite
Upgrade
Work locally with Git: Open your working directory in Postman to keep your API artifacts in sync with your codebase.
Try Native Git
PUT http://localhost:3000/a
No environment
AI
COLLECTIONS
My Collection
http://localhost:3000/api/users/7
Save
New Chat
Start using Agent Mode!
Your plan includes 100 AI credits per month to use Agent Mode.
PUT
http://localhost:3000/api/users/7
Send
Docs Params Authorization Headers 9 Body Scripts Tests Settings
Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
1 {"name": "María Cruz", "email": "maría.cruz@lab.com", "role": "user"}
Body Cookies Headers 7 Test Results
200 OK 7 ms 306 B
{} JSON Preview Visualize
1 {
2 "id": 7,
3 "name": "María Cruz",
4 "email": "maría.cruz@lab.com",
5 "role": "user"
6 }
Add negative-case user tests
Create a small test matrix via runner
Centralize auth header with pre-request script
Describe what you need. Press @ for context.
Auto
ENVIRONMENTS
SPECS
FLOWS
Connect Git Console Terminal 0 0
Globals Vault Tools
25°C
Mostly cloudy
Search
ENG
US
11:31 PM
10/9/2026

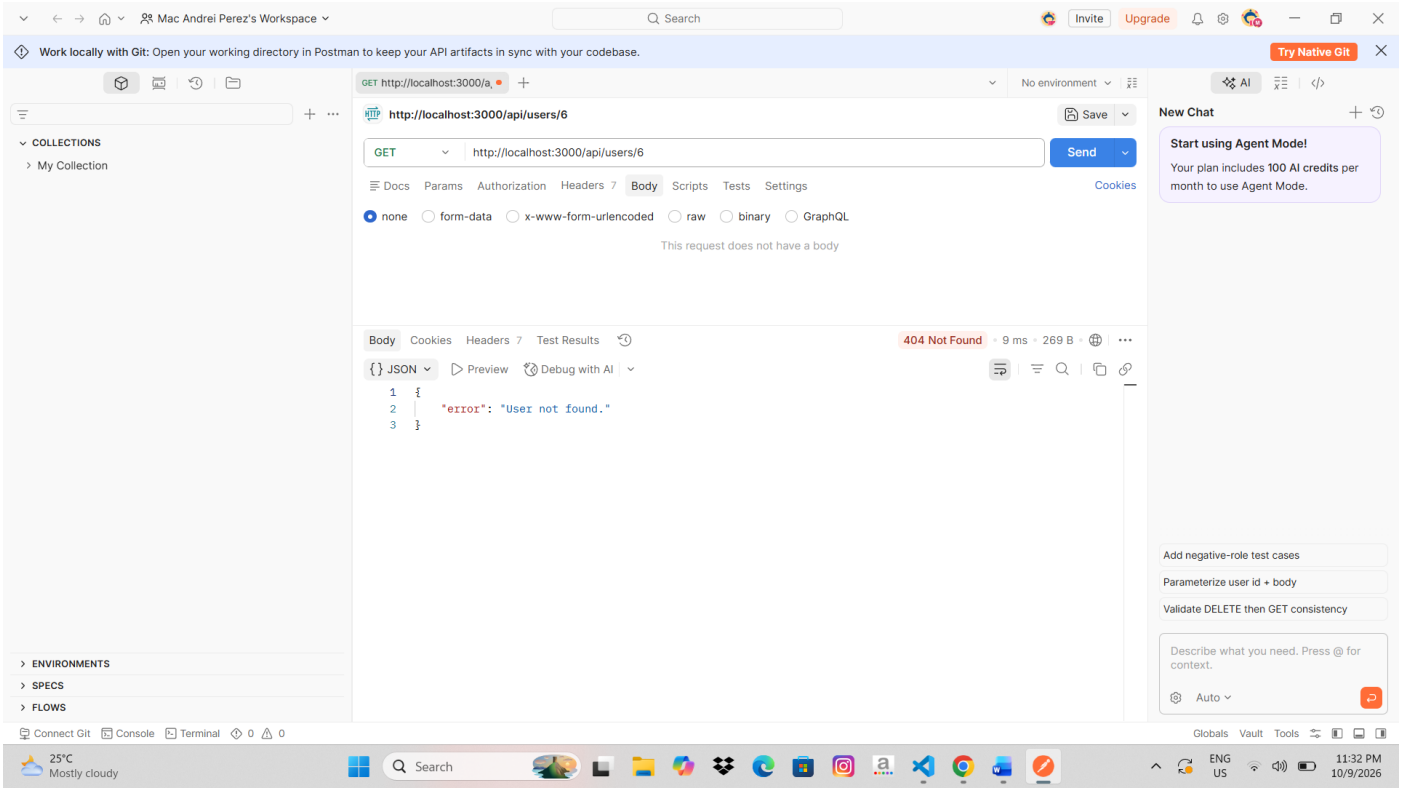
5c — DELETE user

Field	Value
Method	DELETE
URL	http://localhost:3000/api/users/3
Expected Status	204 (No Content)



5d — Verify deletion

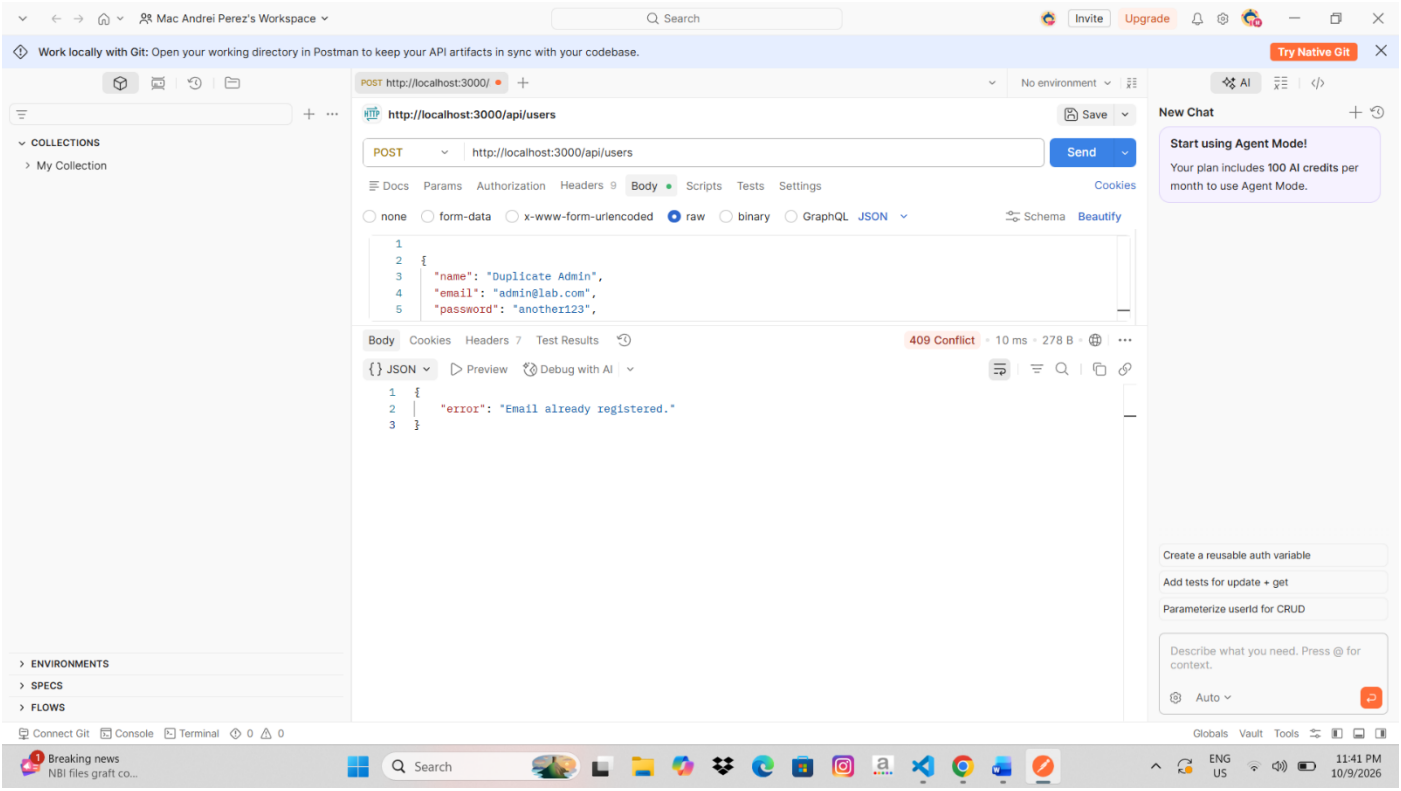
Field	Value
Method	GET
URL	http://localhost:3000/api/users/3
Expected Status	404
Expected Response	{"error": "User not found."}



Deliverable: Screenshots of all four requests.

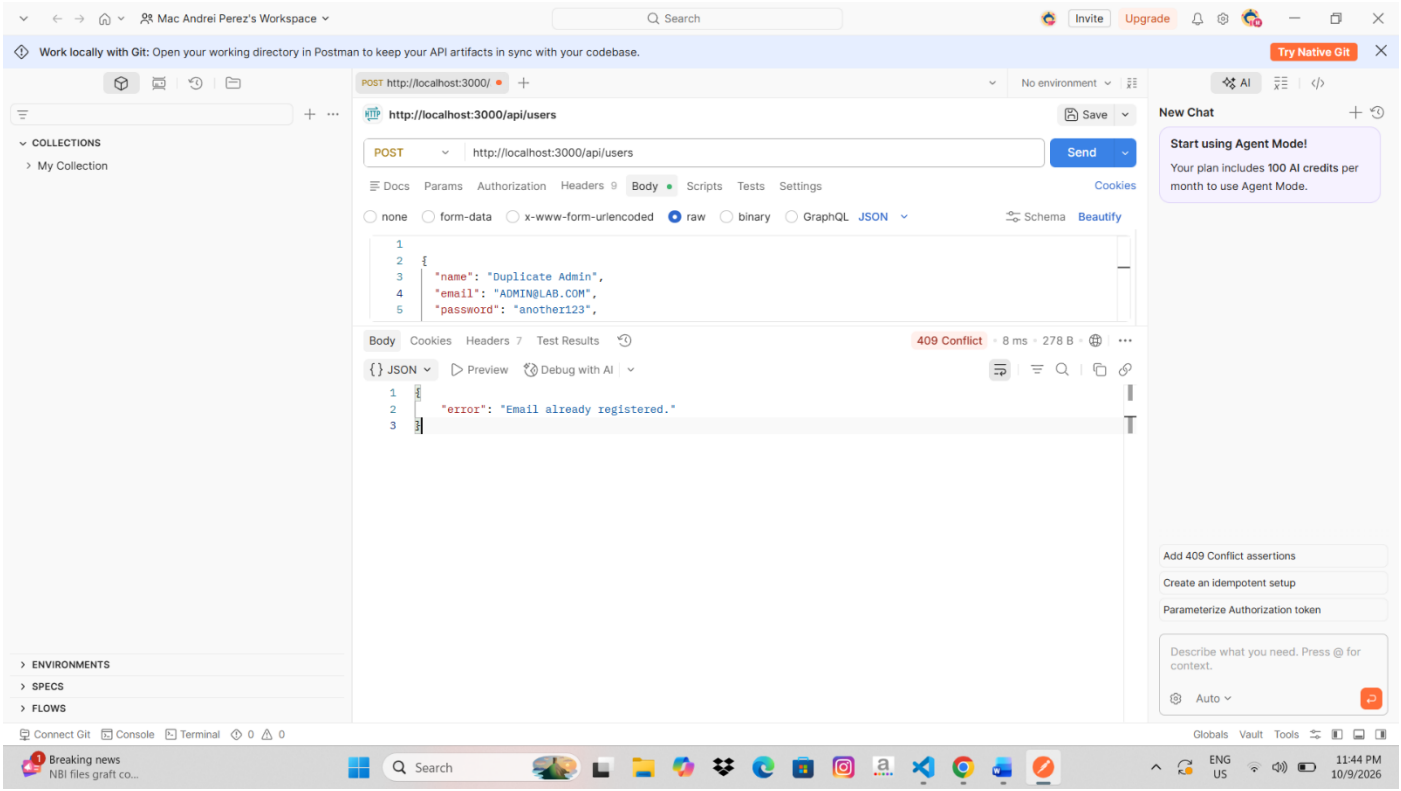
Task 6 — Handle Duplicate Emails (5 pts)

1. Attempt to create another user with admin@lab.com.
    - **Expected:** 409 {"error": "Email already registered."}



2. Attempt with ADMIN@LAB.COM.

- **Expected:** Also 409 (lookups are case-insensitive).

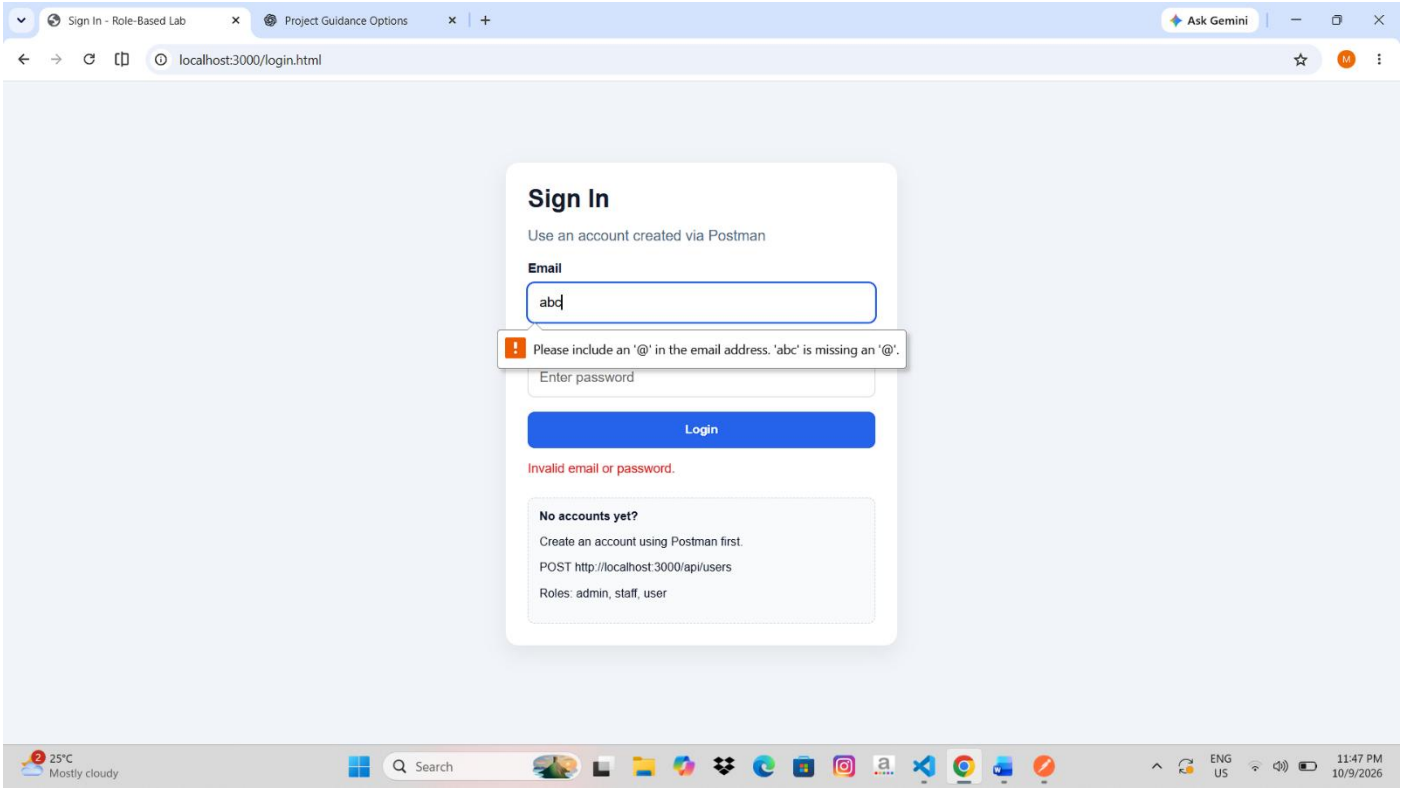


Deliverable: Screenshots of both attempts.

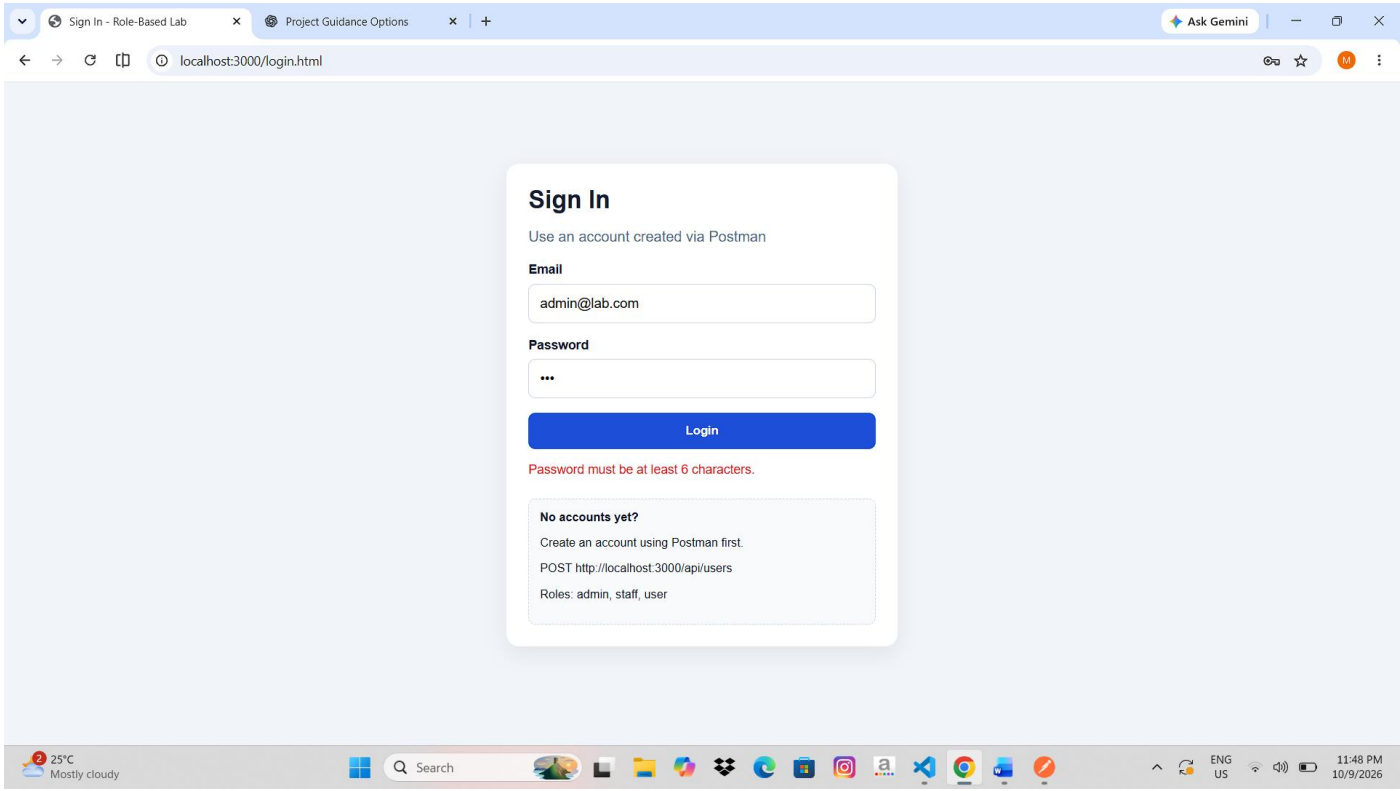
Task 7 — Client-Side Validation (5 pts)

The login page contains client-side validation. Verify:

1. Submit with abc in email — error: "Please enter a valid email address."



2. Submit with a valid email and password 123 — error: "Password must be at least 6 characters."



Deliverable: Screenshots showing both validation errors.

Task 8 — Reflection Questions (10 pts)

Answer the following Questions:

Q1. In server.js, what single block of code controls which dashboard a user sees? Paste it.

Answer:

The POST /api/login route controls which dashboard the user sees. After checking the email, password, and role, the server returns a redirectTo value.

```
const redirectTo =
  user.role === "admin" ? "/admin.html" :
  user.role === "staff" ? "/staff.html" :
  "/user.html";

res.json({
  message: "Login successful",
  user: sanitizeUser(user),
  redirectTo
});
```

Q2. The admin.html page checks sessionStorage.getItem('currentUser'). If a user manually sets sessionStorage.currentUser to {"role": "admin"} using DevTools, will they gain access? What does this reveal about client-side guards?

Answer:

Yes, they may gain access to the admin dashboard's interface if the page only checks the role stored in sessionStorage. This shows that client-side guards are not secure because users can modify browser storage. Real applications must verify permissions on the server.

Q3. Explain the correct order of operations to test the login flow. Why does the order matter?

Answer:

First, start the Express server. Second, create users with different roles through Postman. Third, test the login endpoint using the correct email and password. Finally, open the appropriate dashboard and test incorrect login details. The order matters because users must exist before they can log in, and the server must be running to process requests.

Q4. Why does sanitizeUser strip passwords from every response?

Answer:

The sanitizeUser function removes passwords before user information is returned by the API. This prevents passwords from being exposed in API responses and helps protect users' accounts.

Q5. Propose one change to make this system safer for production use. Explain why.

Answer:
I would use password hashing, such as bcrypt, instead of storing plain-text passwords. Hashing helps protect users' passwords if the user database is exposed. A production system should also enforce authentication and authorization on the server.

Q6. How do GET, POST, PUT, and DELETE support REST API integration? Give one example of each from this lab.
Answer:

- **GET:** Retrieves data. Example: GET /api/users displays all users.
- **POST:** Creates data or submits information. Example: POST /api/users creates a new user.
- **PUT:** Updates existing data. Example: PUT /api/users/1 updates the user with ID 1.
- **DELETE:** Removes data. Example: DELETE /api/users/1 deletes the user with ID 1.

These HTTP methods allow the frontend and backend to communicate using clear, organized operations.

Q7. What is the difference between authentication and authorization in this lab?
Answer:
Authentication verifies who the user is by checking their email and password during login. Authorization determines what the user is allowed to access based on their role, such as admin, staff, or user.

Q8. Why is it important to test the backend first before creating the web frontend?
Answer:
Testing the backend first helps confirm that the API endpoints, validation, login, and CRUD operations work correctly. This makes frontend development easier because the frontend can connect to an API that has already been tested.

Q9. How does this activity demonstrate synchronous communication?
Answer:
The activity demonstrates synchronous communication because the client sends an HTTP request and waits for the server's response before continuing with the result. For example, during login, the frontend waits for the API response before redirecting the user to a dashboard.

Q10. How does JSON help in REST API communication?
Answer:
JSON provides a standard format for sending and receiving structured data between the frontend and backend. In this lab, JSON carries user details when creating accounts and carries login information and results when communicating with the API.

VII. Post-Lab Testing Checklist

Mark / or X next to each.

#	Test Case	Expected Result	Done
1	Server starts with node server.js	Console shows both URLs	/
2	Open http://localhost:3000/login.html	Login form loads with styling	/
3	GET /api/users before creating any user	Returns [] (empty array)	/
4	Create admin via Postman	Status 201, no password in response	/
5	Create staff via Postman	Status 201, role = staff	/
6	Create user via Postman	Status 201, role = user	/
7	GET /api/users	All users listed, no passwords	/
8	Login as admin in Postman	Status 200, redirectTo: "/admin.html"	/
9	Login as staff in Postman	Status 200, redirectTo: "/staff.html"	/
10	Login as user in Postman	Status 200, redirectTo: "/user.html"	/
11	Login with wrong password	Status 401, generic error	/
12	Login with unknown email	Status 401, same generic error	/
13	Login with missing password	Status 400, "Email and password are required."	/

#	Test Case	Expected Result	Done
14	Browser login as admin	Redirect to /admin.html, table lists all users	/
15	Browser login as staff	Redirect to /staff.html	/
16	Browser login as user	Redirect to /user.html	/
17	Logout from any dashboard	sessionStorage cleared, back to login	/
18	Duplicate email creation	Status 409, "Email already registered."	/
19	Duplicate email different casing	Status 409	/
20	Restart server, try logging in	Login fails — data cleared	/
21	Client-side email validation	Error shown, no API call	/
22	Client-side password validation	Error shown, no API call	/
23	GET single user by ID	Status 200, correct user	/
24	PUT update user	Status 200, updated data	/
25	DELETE user	Status 204	/
26	GET deleted user	Status 404	/

VIII. Grading Rubric

Section	Description	Points
Pre-Lab	5 conceptual questions	10
Task 1	Server runs, login page loads	5
Task 2	Users created via Postman (admin, staff, user)	20
Task 3	Login API tested (6 scenarios)	10
Task 4	Browser login flow verified (4 screenshots)	10
Task 5	CRUD operations tested (GET, PUT, DELETE)	20
Task 6	Duplicate email handling verified	5
Task 7	Client-side validation verified	5
Task 8	10 reflection questions	10
Testing Checklist	All 26 items marked correctly	5
Subtotal		**100**
Bonus	Hashed passwords with bcrypt	**+10**

IX. Bonus Task (+10 pts) — Hash Passwords with bcrypt

1. Install bcrypt:

   ```
   npm install bcrypt
   ```

2. Require it at the top of server.js:

   ```
   const bcrypt = require('bcrypt');
   ```

3. In POST /api/users, hash before storing:

   ```
   app.post('/api/users', async (req, res) => {
     // ...validations...
     const hashed = await bcrypt.hash(password, 10);
     const newUser = {
       id: nextId++,
       name,
       email,
       password: hashed,
       role: normalizedRole
     };
     users.push(newUser);
     res.status(201).json(sanitizeUser(newUser));
   });
   ```

4. In POST /api/login, compare with bcrypt:

   ```
   app.post('/api/login', async (req, res) => {
     // ...validations...
     const user = findUserByEmail(email);
     if (!user) return res.status(401).json({ error: 'Invalid email or
   password.' });

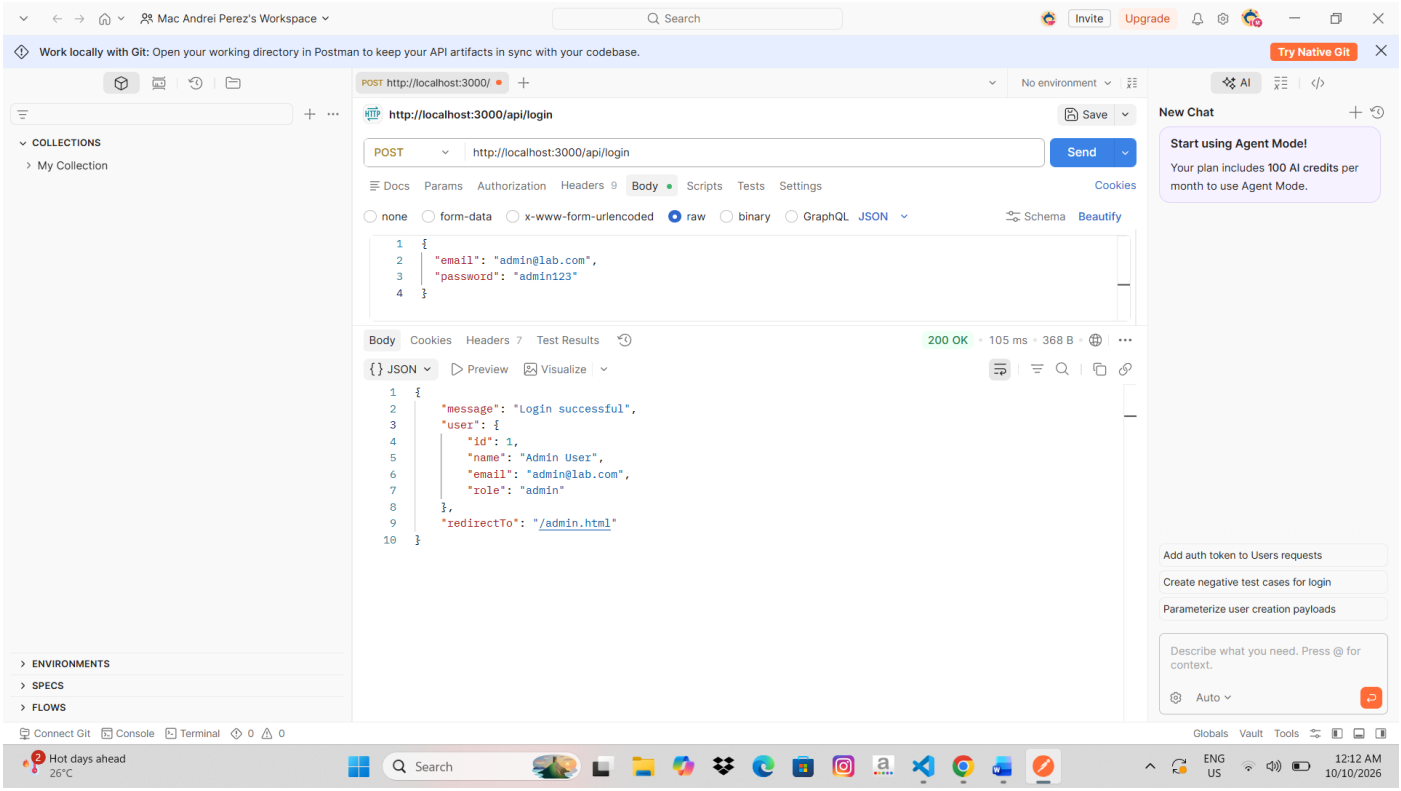
     const match = await bcrypt.compare(password, user.password);
     if (!match) return res.status(401).json({ error: 'Invalid email
   or password.' });

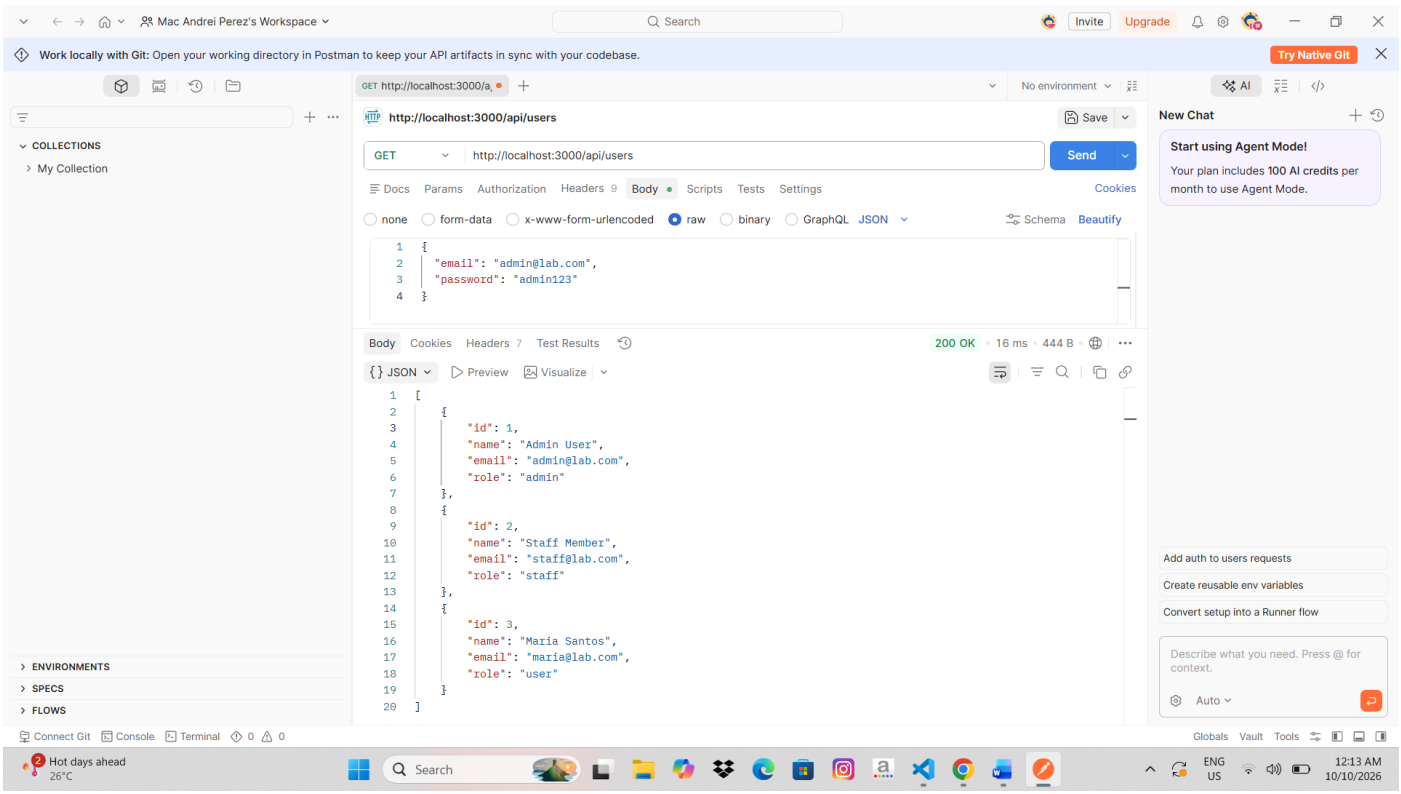
     let redirectTo = '/user.html';
     if (user.role === 'admin') redirectTo = '/admin.html';
     else if (user.role === 'staff') redirectTo = '/staff.html';

     res.json({
       message: 'Login successful',
       user: sanitizeUser(user),
       redirectTo
     });
   });
   ```

5. **Recreate all users in Postman** — old plain-text records will no longer match.
6. Confirm login still works and that GET /api/users never reveals hashes.

Deliverable: Updated server.js + screenshot showing login succeeds after hashing.





X. Submission Requirements

Submit a **ZIP file** or **GitHub repository link** containing:

1. server.js
2. package.json
3. public/login.html
4. public/admin.html
5. public/staff.html
6. public/user.html
7. public/style.css

8. Printed Answers and Screenshots:
    - Pre-lab answers
    - Reflection answers
    - Testing checklist (marked)
    - Screenshots of Postman tests (GET, POST, PUT, DELETE)
    - Screenshot of web login page
    - Screenshot of admin dashboard after admin login
    - Screenshot of staff dashboard after staff login
    - Screenshot of user dashboard after user login
    - Screenshot showing duplicate email error
    - Screenshot showing client-side validation errors

XI. Debugging Table

Problem	Possible Cause	Solution
Server not starting	Missing package	Run npm install
401 Unauthorized	Invalid credentials	Use correct email/password
CORS error	CORS not enabled	Add app.use(cors()) if needed
Login fails	Wrong password	Use admin123, staff123, or maria123
Redirect not working	redirectTo missing	Check server response in Postman
Users not showing	Server restarted	Recreate users via Postman
sessionStorage not set	Fetch error	Check browser console for errors

XII. Summary

This laboratory activity demonstrated:

- **REST API fundamentals** — building a Node.js + Express backend with full CRUD operations
- **In-memory data storage** — understanding the limitations of non-persistent storage
- **Role-based access** — implementing admin, staff, and user roles with different dashboards
- **Server-side authentication decisions** — the server controls redirects based on role
- **Client-side integration** — consuming the REST API from a web login page using fetch()
- **Postman testing** — validating every endpoint before building the frontend
- **Security basics** — password sanitization, generic error messages, and client-side guards

This activity directly supports Module 2 topics: Web services and APIs (REST), synchronous communication, JSON data format, authentication and security basics, and client-server integration.